

Syntactic Implicit Parameters with Static Overloading

Microsoft Research Technical Report, MSR-TR-2025-56, 2026-01-22 (v4)

DAAN LEIJEN, Microsoft Research, USA

TIM WHITING, Brigham Young University, USA

Implicits provide a powerful mechanism for term-based inference, where “obvious” arguments can be omitted and inferred by the type checker. This can greatly reduce the programmer’s burden and improve the clarity of expression. As such, many languages support a form of implicits in practice, such as type classes in Haskell or Lean, or implicits in Scala. Unfortunately, many of these systems have become increasingly complex and often require significant implementation effort.

In this paper we take a fresh look at the design space with an arguably simpler approach based on two orthogonal features: *syntactic implicit parameters* and *static overloading*. Each of these features is limited in scope and has a straightforward implementation. Taken together though, they are surprisingly expressive and we believe they can cover many of the common usage scenarios of implicits in practice.

We formalize our system and provide various examples, and prove our elaboration is coherent. We also give an inference algorithm and show it is sound and complete. Our system is fully implemented in the Koka language, and we describe our experience with these features at scale, and discuss further extensions.

1 INTRODUCTION

Implicits provide a powerful mechanism for term-based inference, where “obvious” terms can be omitted and inferred by the type checker. This can greatly reduce the programmers burden and improve the clarity of expression. For example, without implicits, a programmer may have to write `show_list show_int []` to show a list of integers, while a Haskell programmer instead can simply write `show []`, where the correct type class dictionary for showing lists of integers is provided implicitly.

As such, many languages support a form of implicits in practice, such as type classes in Haskell or Lean, implicits in Scala, or in systems like Rocq and Agda where the language can provide obvious proof terms [Devriese and Piessens 2011; Odersky et al. 2017; Selsam et al. 2020; Sozeau and Oury 2008; P. Wadler and Blott 1989]. However, the design of many such systems has become increasingly complex and difficult to implement, and it is not always clear how the different approaches in all these systems relate to each other [B. C. Oliveira et al. 2010].

In this paper we take a fresh look at the design space with an arguably simpler approach based on two orthogonal features: *syntactic implicit parameters* and *static overloading*. Each of these features is limited in scope and has a straightforward implementation. Taken together though, they are surprisingly expressive and we believe they can cover many of the common usage scenarios of implicits in practice. The first idea is to treat implicit parameters syntactically as a form of dynamic binding:

Syntactic implicit parameters are parameter names that are supplied literally as arguments at the call site.

For example, we can define a function with an implicit parameter `base` as:

```
fun show-int( x : int, ?base : int ) : string
  if x < base
  then show-digit(x)
  else show-int(x / base) ++ show-digit(x % base)
```

This function takes an implicit parameter `base`, denoted by the question mark, which can be used inside the function as any regular parameter (like `x < base`). When calling `show-int` though, we do not have to provide the `?base` parameter explicitly and can leave it out: at each call site the name

base is syntactically used as the argument, and resolved *in the scope of the call site*. For example,

```
val base = 10 in show-int(42)
```

elaborates to

```
val base = 10 in show-int(42,base)
```

which evaluates to "42". We see this elaboration as well in the `show-int` function itself where the expression `show-int(x / base)` elaborates to `show-int(x / base,base)` where it passes the implicit parameter `base` again to the recursive call.

This view of implicit parameters as a form of dynamic binding (but with a static declaration!) is quite straightforward and seems almost too simple to be of much use. However, it turns out to be quite expressive in combination with *static overloading*. This brings us to the second idea, where we define static overloading purely as a form of automatic qualification:

Static overloading elaborates plain names to fully qualified names based on the local type context.

The essence of this idea was first described by Leijen and Ye [2025] as an application of type inference under a prefix. First, we allow functions to be declared with qualified names, for example:

```
fun int/show( x : int ) : string
  show-int(x,10)

fun float/show( f : float64 ) : string
  show-float(f)
```

(where `int/` and `float/` can be arbitrary “module” names). Of course, such qualified names already occur naturally as well in most languages when different modules are imported that export the same (unqualified) name. We now allow the programmer to write an unqualified `show` and have it be resolved to either definition based on the local type context. For example `show(1)` is elaborated to the fully qualified `int/show(1)` based on the (static) type of the argument. This is already quite convenient in practice, and is again a simple mechanism that is straightforward to implement – for example the C language implements this form of static overloading for many common math operations. However, static overloading by itself is quite limited as it does not allow for *abstraction*. For example, consider a `show` function for lists:

```
fun list/show( xs : list<a> ) : string
  match xs
  Cons(x,xx) -> show(x) ++ ":@" ++ list/show(xx) // rejected
  Nil        -> "[]"
```

This is rejected since we cannot at this point statically resolve which `show` function is required for the `show(x)` expression (as the type of the list elements is polymorphic). Here is where we can now use our new syntactic implicit parameters to delay resolving which particular `show` to use:

```
fun list/show( xs : list<a>, ?show : a -> string ) : string
  match xs
  Cons(x,xx) -> show(x) ++ ":@" ++ list/show(xx)
  Nil        -> "[]"
```

Just like a Haskell programmer, we can now write `show([1])` to show a list of integers.

- Static overloading first elaborates the plain `show` to `list/show([1])`, based on the type `list<int>` of the argument.
 - Subsequently, the `?show` implicit parameter is now supplied as `list/show([1], show)`.
 - Static overloading kicks in again and further disambiguates this (implicit) `show` to `int/show` (again based on the local type context), which results in the final elaboration as `list/show([1], int/show)`.
- As an aside, in the recursive call in the `list/show` function, the implicit parameter is also supplied as `list/show(xx, show)`; this does not need further disambiguation though as it is already bound locally.

Note also that an expression like `show([])` is still rejected as the element type cannot be statically determined (similar to unresolved overloading in Haskell). Here we either need to give an explicit type signature, or provide the implicit parameter ourselves (as `show([], int/show)` for example).

As we could see, the resolving of syntactic overloads and syntactic implicit parameters is applied recursively. For example, we can show lists of lists of integers as `show([[1], [2]])`, which elaborates to `list/show([[1], [2]], fn(xs) list/show(xs, int/show))` (where the implicit `list/show` argument is automatically eta-expanded in order to supply its own implicit argument).

Even though syntactic implicits and static overloading by themselves are straightforward features, this recursive interaction between them makes them quite expressive. We believe that they can handle many situations that are usually addressed using more elaborate language extensions. Also, our new features both are source-to-source elaborations and a programmer can always use unambiguous fully qualified names, or provide implicit parameters explicitly. As a pure elaboration, there are no new semantic features (like an “instance”, or special “implicit scopes” etc.) – it is all just names, functions, and parameters. We make the following contributions:

- We formalize syntactic implicit parameters and static overloading precisely (Section 2). The type rules are specified using inference under prefix [Leijen and Ye 2025] to give unambiguous type contexts which are necessary for static overloading. Our rules go beyond the original example of static overloading by Leijen and Ye [2025] in that we study in particular its interaction with syntactic implicit parameters and how we maintain coherence and stability (Section 2.10).
- We show that even though our two ideas look innocent enough, together they make general type checking undecidable where we can encode a Turing machine on the type level which is executed by the type checker (Section 3). We give an improved set of “finite” rules that recover decidability at the price of giving up completeness. We show though that our new finite rules are still sound with respect to our original system.
- We give an inference algorithm for the finite rules (Section 3.4) and show it is sound and complete. By specifying the algorithm under an effect handler we can modularly optimize the algorithm to not explore unnecessary branches in the search space.
- Our system is fully implemented in the Koka language [Leijen 2019 2021] and we discuss various implementation aspects of using these features at scale (Section 4).
- We also discuss the extension to *phantom implicits* where the compiler can resolve special implicit parameters in other ways than by name. For example `?kk-line:int` may provide the line number at the call site. Taking this further, we show how divergence constraints in the Koka language [Leijen 2014] can be handled by phantom implicits instead (Section 4.3).

Proofs and appendices can be found in the supplementary material.

2 FORMALIZATION

We start with formalizing both syntactic implicits and static overloading within a single calculus.

2.1 Syntax

Figure 1 gives the syntax of our core calculus that has qualified names. A plain name is written as x while a fully qualified name is written as z (or m for modules when needed). We use the *hat* operator to unqualify a name. For example, if a qualified name z has the form $x_1/. \dots /x_n/x$, then $\hat{z} = x$. All bindings can be specified with a qualified name z , and we can write $\lambda foo/x. foo/x + 1$ for example.

For simplicity, our calculus does not make implicit parameters first-class and we can only bind them at let bindings. A let binding binds a *term* t which can start with a sequence of implicit parameter bindings, followed by a regular expression e . For example,

```
let mod/plus =  $\lambda?mod. \lambda x. \lambda y. (x + y) \% mod$ 
in (let mod = 8 in mod/plus 5 6)
```

z, m	$::= m/x$	(qualified name)	τ	$::= \alpha$	(type variable)
	x	(plain name)		$\tau \rightarrow \tau$	(function arrow)
				<code>int</code> <code>bool</code> ...	(type constants)
e	$::= z$	(variable)	ρ	$::= ?x:\tau \rightarrow \rho$	(implicit arrow)
	$e e$	(application)		τ	(mono type)
	$\lambda z. e$	(function)	σ	$::= \forall \alpha. \sigma \mid \rho$	(type scheme))
	$\text{let } z = t \text{ in } e$	(let binding)			
t	$::= \lambda ?z. t$	(implicit param)	Γ	$::= z_1 : \sigma_1, \dots, z_n : \sigma_n$	(type env.)
	e	(expression)	Q	$::= \{\alpha_1 = \tau_1, \dots, \alpha_n = \tau_n\}$	(prefix)
$\hat{m}/x$	$= x$	(unqualify)			
$\hat{x}$	$= x$				

Fig. 1. Syntax of types and terms.

binds an implicit parameter *mod* (and evaluates to 3).

Expressions have a monomorphic type τ which are either type variables α , or function types $\tau_1 \rightarrow \tau_2$. We also use types like *int* or *bool* in examples. Let-bound values are assigned a polymorphic *type scheme* σ . Since let-bound values can have implicit parameters, we extend the standard type schemes with implicit parameter types ρ . An implicit parameter type $?x:\tau \rightarrow \rho$ denotes a function type that takes an implicit parameter x of type τ . The type of an implicit parameter binds stronger than the function arrow, so $?x:\tau \rightarrow \tau$ should be read as $(?x:\tau) \rightarrow \tau$. A type scheme is generally of the form $\forall \bar{\alpha}. ?x_1:\tau_1 \rightarrow \dots ?x_n:\tau_n \rightarrow \tau$ and at every variable occurrence we fully instantiate the type variables $\bar{\alpha}$ and supply all implicit parameters x_1 to x_n .

2.2 Type Inference under a Prefix

For the purpose of static overloading, we formulate our rules as type rules under a prefix Q [Leijen and Ye 2025]. This allows us to specify deterministic type rules with only principal derivations. For example, consider $\lambda x. \text{show } x$ – we should reject this as it is ambiguous which *show* function to elaborate to. However, when using standard Hindley-Milner rules one is allowed to use more specific types in derivations, and for example assume type *int* or *bool* for the parameter which makes it well-typed. With type inference under a prefix we avoid this issue. Moreover, as shown by Leijen and Ye [2025], specifying the type rules under a prefix also allows us to “read off” the inference algorithm from the rules directly, while still being close to the clarity of standard Hindley-Milner style type rules. We only briefly describe the essential details of prefixes and refer the interested reader to the original work.

A prefix Q is essentially just a set of type variable bounds $\alpha = \tau$. In general a prefix is a collection of such bounds and can be inconsistent or have duplicate bindings, like $\{\alpha = \beta \rightarrow \text{int}, \alpha = \text{int} \rightarrow \gamma\}$ or $\{\alpha = \text{bool}, \alpha = \text{int}\}$. We write $\theta \models Q$ if a substitution θ is a solution to Q that satisfies all constraints (with $\forall (\alpha = \tau) \in Q. \theta \alpha = \theta \tau$). If there exists any solution, we call Q consistent and write just $\models Q$.

It turns out for any consistent Q there is also a least (or best) solution which we call the prefix solution written as $\langle Q \rangle$. We often write $Q[\tau]$ as a shorthand for applying the prefix solution as $\langle Q \rangle(\tau)$. Finally, two prefixes are equivalent whenever their solution substitutions are equivalent where $Q_1 \equiv Q_2 \Leftrightarrow \langle Q_1 \rangle \equiv \langle Q_2 \rangle$. For example, we have $\{\alpha = \beta \rightarrow \text{int}, \alpha = \gamma \rightarrow \gamma\} \equiv \{\gamma = \text{int}, \beta = \gamma, \alpha = \gamma \rightarrow \gamma\} \equiv \{\beta = \text{int}, \gamma = \text{int}, \alpha = \text{int} \rightarrow \text{int}\}$. Similar to α -renaming, we can always substitute equivalent prefixes in type derivations.

$$\begin{array}{c}
\boxed{
\begin{array}{ccc}
Q \vdash \tau \approx \tau & & \\
\downarrow & \uparrow & \uparrow \\
\text{out} & \text{in} & \text{in}
\end{array}
}
\end{array}
\quad
\frac{\alpha \notin \text{ftv}(\tau)}{\{\alpha=\tau\} \vdash \alpha \approx \tau} \text{EQ-VAR}
\quad
\frac{}{\emptyset \vdash \tau \approx \tau} \text{EQ-ID}$$

$$\frac{Q_1 \vdash \tau_1 \approx \tau_3 \quad Q_2 \vdash \tau_2 \approx \tau_4}{Q_1, Q_2 \vdash \tau_1 \rightarrow \tau_2 \approx \tau_3 \rightarrow \tau_4} \text{EQ-FUN}
\quad
\frac{Q \vdash \tau_2 \approx \tau_1}{Q \vdash \tau_1 \approx \tau_2} \text{EQ-REFL}$$

Fig. 2. Type equivalence.

2.3 Type Equivalence under a Prefix

A *consistent union* is written as Q_1, Q_2 and denotes the union $Q_1 \cup Q_2$ where $Q_1 \cup Q_2$ is solvable. We use this in the conclusion of derivation rules to concisely denote that we can only derive consistent prefixes. This helps us write nice declarative type rules while avoiding having to thread a substitution linearly through each sub-derivation. We see this when defining type equivalence as shown in Figure 2.

A rule $Q \vdash \tau_1 \approx \tau_2$ states that a type τ_1 is equal to a type τ_2 under a (result) prefix Q . In particular, in the rule [EQ-FUN] we can easily compose the prefixes Q_1 and Q_2 from each sub derivation. These definitions are sound and complete [Leijen and Ye 2024]:

Theorem 2.1. (Type equivalence under a prefix is sound)

If $Q \vdash \tau_1 \approx \tau_2$ then $Q[\tau_1] = Q[\tau_2]$.

Theorem 2.2. (Type equivalence under prefix is complete)

If $\theta\tau_1 = \theta\tau_2$, then there exists a Q such that $Q \vdash \tau_1 \approx \tau_2$ and $Q \sqsubseteq \theta$.

Soundness states that if we can derive that τ_1 and τ_2 are equivalent under a prefix Q , then the types are syntactically equal under the prefix solution: $Q[\tau_1] = Q[\tau_2]$. Completeness shows that if there exists any substitution θ that makes two types equal, then we can also derive that these types are equivalent under a prefix Q , and that this prefix is also the “best” (most-general) solution: $\langle Q \rangle \sqsubseteq \theta$.

The prefix Q gives us an elegant way to declaratively specify type rules where we are assured that we can still implement this efficiently in a compiler using a linear substitution. In the case of the type equivalence rules, this corresponds directly to the usual unification algorithm [Pierce [2002], §22.4.5].

2.4 Instantiation

For most of the type checking rules, we are using essentially the bidirectional inference rules under a prefix as given by Leijen and Ye [2025]. For now, we look in particular at the checking rules for resolving variables:

$$\frac{z : \sigma \in \Gamma \quad Q \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow z \bar{e}}{Q \mid \Gamma \Vdash z \dot{:} \tau \rightsquigarrow z \bar{e}} \text{VAR-DIRECT}$$

Here the typing judgement $Q \mid \Gamma \Vdash z \dot{:} \tau \rightsquigarrow z \bar{e}$ states that a variable z can be *checked* ($\dot{:}$) to have type τ in a type environment Γ under the (result) prefix Q , and returning an elaborated expression $z \bar{e}$ (where $\bar{e}$ is a list of potentially instantiated implicit parameters). In particular, we must have $z : \sigma \in \Gamma$, and we must be able to *instantiate* the type scheme σ to τ , written as $Q \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$.

Figure 3 shows the rules for type scheme instantiation. The rule [INST-MONO] says that two mono types need to be equivalent $Q \vdash \tau_1 \approx \tau_2$, i.e. they need to be unifiable under Q . The rule [INST-QUANTIFY] is also standard and instantiates a quantifier using a fresh type variable α .

$$\begin{array}{c}
\boxed{
\begin{array}{ccccc}
Q & | & \Gamma \vdash \sigma \sqsubseteq \tau & \rightsquigarrow & \bar{e} \\
\downarrow & \uparrow & \uparrow & \uparrow & \downarrow \\
\text{out} & \text{in} & \text{in} & \text{in} & \text{out}
\end{array}
} \\
\\
\frac{Q_1 \mid \Gamma \vdash \rho \sqsubseteq \tau_2 \rightsquigarrow \bar{e} \quad Q_2 \mid \Gamma \Vdash x \vdash Q_1[\tau_1] \rightsquigarrow e}{Q_1, Q_2 \mid \Gamma \vdash ?x:\tau_1 \rightarrow \rho \sqsubseteq \tau_2 \rightsquigarrow e \bar{e}} \text{INST-IMPLICIT} \\
\\
\frac{Q \vdash \tau_1 \approx \tau_2}{Q \mid \Gamma \vdash \tau_1 \sqsubseteq \tau_2 \rightsquigarrow \cdot} \text{INST-MONO} \quad \frac{Q \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e} \quad \text{fresh } \alpha}{Q \mid \Gamma \vdash \forall \alpha. \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}} \text{INST-QUANTIFY}
\end{array}$$

Fig. 3. Type scheme instantiation.

$$\begin{array}{c}
\boxed{
\begin{array}{ccccc}
Q & | & \Gamma \Vdash z \vdash \tau & \rightsquigarrow & e \\
\downarrow & \uparrow & \uparrow & \uparrow & \downarrow \\
\text{out} & \text{in} & \text{in} & \text{in} & \text{out}
\end{array}
} \\
\\
\frac{z:\sigma \in \Gamma \quad Q \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}}{Q \mid \Gamma \Vdash z \vdash \tau \rightsquigarrow z \bar{e}} \text{VAR-DIRECT} \\
\\
\frac{z \notin \text{dom}(\Gamma) \quad m/z:\sigma \in \Gamma \quad Q \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}}{Q \mid \Gamma \Vdash z \vdash \tau \rightsquigarrow m/z \bar{e}} \text{VAR-QUALIFY}
\end{array}$$

Fig. 4. Variable rules.

2.5 Syntactic Implicit Parameters

For standard Hindley-Milner type schemes the previous rules suffice, but in our case we added one more rule: the [INST-IMPLICIT] rule instantiates an implicit parameter type $?x:\tau_1 \rightarrow \rho$. In this case we can use the type rule for variables (recursively) to resolve the literal name x as:

$$Q_2 \mid \Gamma \Vdash x \vdash Q_1[\tau_1] \rightsquigarrow e$$

This rule concisely captures the essence of syntactic implicit parameters: we resolve the name of an implicit parameter $?x$ (recursively) in the local scope Γ .

Note that we first instantiate $Q_1 \mid \Gamma \vdash \rho \sqsubseteq \tau_2$ which returns the prefix Q_1 , and then propagate this information to check x under $Q_1[\tau_1]$. This is important as the regular parameters often give rise to further type constraints that can help to resolve the implicit parameters unambiguously. We will show some examples after discussing static overloading.

Furthermore, each resolved implicit parameter is elaborated as $\bar{e}$ and added to the list of implicit parameters that need to be applied (as $e \bar{e}$). If we look at the example at the start of this section, we can derive for the *mod/plus* expression:

$$\begin{array}{c}
\frac{\frac{\frac{\emptyset \vdash \tau \approx \tau}{\emptyset \mid \Gamma \vdash \tau \sqsubseteq \tau \rightsquigarrow \cdot} \quad \frac{\frac{\text{mod}:\text{int} \in \Gamma \quad \emptyset \vdash \text{int} \approx \text{int}}{\emptyset \mid \Gamma \vdash \text{int} \sqsubseteq \text{int} \rightsquigarrow \cdot}}{\emptyset \mid \Gamma \Vdash \text{mod} \vdash \emptyset[\text{int}] \rightsquigarrow \text{mod}}}{\emptyset \mid \Gamma \vdash ?\text{mod}:\text{int} \rightarrow \tau \sqsubseteq \tau \rightsquigarrow \text{mod}} \quad \text{mod/plus}:\text{?mod:int} \rightarrow \tau \in \Gamma \\
\hline
\emptyset \mid \Gamma \Vdash \text{mod/plus} \vdash \tau \rightsquigarrow \text{mod/plus mod}
\end{array}$$

where we use $\tau = \text{int} \rightarrow \text{int} \rightarrow \text{int}$ and $\Gamma = \{\text{mod/plus}:\text{?mod:int} \rightarrow \tau, \text{mod}:\text{int}\}$. As we can see, the variable *mod/plus* is now elaborated to *mod/plus mod* where the implicit parameter is passed to the *mod/plus* function.

2.6 Static Overloading as Qualification

For our notion of static overloading we require another variable rule. In particular, when we have a variable occurrence z that is not occurring exactly in the type environment Γ , we can attempt to find a qualified name m/z that matches the required type. Figure 4 shows the new variable rule

[VAR-QUALIFY] (together with the earlier [VAR-DIRECT] rule):

$$\frac{z \notin \text{dom}(\Gamma) \quad m/z : \sigma \in \Gamma \quad Q \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}}{Q \mid \Gamma \vdash z : \tau \rightsquigarrow m/z \bar{e}} \text{VAR-QUALIFY}$$

The first pre-condition states that z cannot occur directly in Γ , such that applying [VAR-DIRECT] or [VAR-QUALIFY] is deterministic. We can satisfy the other conditions if there exists an m with $m/z : \sigma$ in Γ such that we can instantiate σ to the required type τ , giving an elaborated expression $m/z \bar{e}$. Note that we keep the [INST-IMPLICIT] rule (in Figure 3) unchanged such that any implicit parameters can now also be resolved to a qualified name with the [VAR-QUALIFY] rule.

However, the rule as stated introduces *incoherence*: there might be multiple m with m/z in Γ that can be instantiated to the required type, each elaborating to a different expression. Which one to choose? In our case we eliminate incoherence by construction. The main type inference rule for variables requires a unique elaboration:

$$\frac{\exists! e. \quad Q \mid \Gamma \vdash z : \tau \rightsquigarrow e}{Q \mid \Gamma \vdash z : \tau} \text{VAR}$$

where we use the notation $\exists! e$ for “there exists a unique e ”.

These rules concisely capture the essence of static overloading: we can elaborate a name z to a fully qualified name m/z as long as the solution is unique for a given type context τ .

Although each of these rules is quite simple, the recursive nature of the rules [VAR-QUALIFY] and [INST-IMPLICIT] make it quite expressive. Note in particular that the elaboration from z to m/z may include further implicit parameters $\bar{e}$ that were resolved recursively. Consider for example the following type environment Γ :

$\text{int}/\text{show} : \text{int} \rightarrow \text{string}$, $\text{list}/\text{show} : \forall \alpha. ?\text{show} : (\alpha \rightarrow \text{string}) \rightarrow \text{list } \alpha \rightarrow \text{string}$

When we type check the expression $\text{show } [1, 2]$ we can proceed as:

$$\frac{\frac{\frac{\frac{\frac{\{\alpha=\text{int}\} \vdash \text{list } \alpha \rightarrow \text{string} \approx \text{list } \text{int} \rightarrow \text{string}}{\{\alpha=\text{int}\} \mid \Gamma \vdash \text{list } \alpha \rightarrow \text{string} \sqsubseteq \text{list } \text{int} \rightarrow \text{string} \rightsquigarrow \cdot}}{\{\alpha=\text{int}\} \mid \Gamma \vdash ?\text{show} : (\alpha \rightarrow \text{string}) \rightarrow \text{list } \alpha \rightarrow \text{string} \sqsubseteq \text{list } \text{int} \rightarrow \text{string} \rightsquigarrow \text{int}/\text{show}}}{\text{list}/\text{show} : ?\text{show} : (\alpha \rightarrow \text{string}) \rightarrow \text{list } \alpha \rightarrow \text{string} \in \Gamma}{\{\alpha=\text{int}\} \mid \Gamma \vdash \forall \alpha. ?\text{show} : (\alpha \rightarrow \text{string}) \rightarrow \text{list } \alpha \rightarrow \text{string} \sqsubseteq \text{list } \text{int} \rightarrow \text{string} \rightsquigarrow \text{int}/\text{show}}}{\exists! \{\alpha=\text{int}\} \mid \Gamma \vdash \text{show} : \text{list } \text{int} \rightarrow \text{string} \rightsquigarrow \text{list}/\text{show } \text{int}/\text{show}} \text{VAR}$$

where the elaborated list/show is passed its implicit parameter as int/show (which is itself elaborated from $?\text{show}$). Here we also see the importance of applying Q_1 (as $Q_1[\tau_1]$) in the [INST-IMPLICIT] rule: in the above derivation this propagates the element type α as $\{\alpha=\text{int}\}[\alpha \rightarrow \text{string}] = \text{int} \rightarrow \text{string}$ to resolve the implicit $?\text{show}$ parameter recursively in the [VAR-QUALIFY] rule – where it can now be uniquely elaborated to int/show . As another example, the expression $\text{show } [1, 2]$ from the introduction is recursively elaborated to $\text{list}/\text{show } (\text{list}/\text{show } \text{int}/\text{show}) [1], [2]$. As an aside, we do not need the eta-expansion here as shown in the introduction, since in our calculus all implicit parameters come first and thus we can use partial applications instead.

2.7 Implicit Parameter Binding

Before we give the full type inference rules, we first consider the inference ($\vdash$) rule for implicit bindings:

$$\frac{Q \mid \Gamma, z : \alpha \vdash t \overset{?}{\vdash} \rho \quad \text{fresh } \alpha}{Q \mid \Gamma \vdash \lambda z. t \overset{?}{\vdash} ?\hat{z} : \alpha \rightarrow \rho} \text{IMPLICIT}$$

Just like a regular lambda binding, the name z is added to the environment Γ and is in scope when checking the body t . Following the rules for type inference under a prefix, we assign an abstract type α to z which can be bound in the result prefix Q (if so required).

Even though we can bind an implicit parameter with a qualified name z , we still always use a unqualified plain name $\hat{z}$ in the type: at each call site we always look for the unqualified name. This is important when using multiple implicit parameters of the same name. Consider for example a *show* function for tuples:

let *tuple/show* = $\lambda ?fst/show. \lambda ?snd/show. \lambda x. \text{fst/show } (fst \ x) \ \# \text{, " } \# \text{snd/show } (snd \ x)$
in *show* (1, True)

where *tuple/show* has type $\forall \alpha \beta. ?show : (\alpha \rightarrow string) \rightarrow ?show : (\beta \rightarrow string) \rightarrow (\alpha, \beta) \rightarrow string$. Here we use qualified implicit parameters to distinguish showing each component of the tuple in the body, while at a call site we still resolve each one individually as a plain *show*.

The *show* function for tuples nicely illustrates how static overloading and implicit parameters can really help in practice: consider an expression like *show* [(1), True] which is automatically elaborated to *list/show* (*tuple/show* (*list/show* *int/show* *bool/show*) [(1), True]) – significantly reducing the programmer’s burden.

2.8 Full Type Rules

Figure 5 gives the full type rules under a prefix. These are based mostly on the rules given by Leijen and Ye [2025] and we refer to that work for in-depth discussion on the design. For our purposes, the variable and instantiation rules are new, and we added the [IMPLICIT] rule. For clarity, the rules are stated without rewriting to an elaborated expression, but sometimes we make this explicit. In particular, the full [VAR] rule would be:

$$\frac{\exists ! e. Q \mid \Gamma \Vdash z \overset{?}{\vdash} \tau \rightsquigarrow e}{Q \mid \Gamma \vdash z \overset{?}{\vdash} \tau \rightsquigarrow e} \text{VAR}$$

where it elaborates z to e . For all other rules the elaboration is trivial as only the [VAR] rule qualifies names and adds implicit parameters – all other expressions just propagate the elaboration of their components in the obvious way. For example, the full rule for [FUN] with elaboration is:

$$\frac{Q \mid \Gamma, z : \tau_1 \vdash e \overset{?}{\vdash} \tau_2 \rightsquigarrow e'}{Q \mid \Gamma \vdash \lambda z. e \overset{?}{\vdash} \tau_1 \rightarrow \tau_2 \rightsquigarrow \lambda z. e'} \text{FUN}$$

An interesting aspect is that almost all rules are checking rules ($\overset{?}{\vdash}$) with just three exceptions for [INF], [GEN], and [IMPLICIT] which are inference rules ($\vdash$). In the [LET] rule, the type of the binding x must be inferred and we use [GEN] to infer a most general type, which uses in turn [INF] and [IMPLICIT] to infer a mono type. The [INF] rule infers a type by switching to checking if the expression can be typed with a fresh monotype α .

The [APP-ARG] rule types applications where first the function expression is checked as $\alpha \rightarrow \tau$, and then the argument is checked where we propagate the discovered type information as $Q_1[\alpha]$. This order does not always work though as often we apply an overloaded variable and we need to first check the argument expression to discover enough type information to disambiguate the variable. Consider for example *show* [1]. This is where the [APP-VAR] rule comes in. This rule is always preferred over [APP-ARG] when it can apply, and it matches syntactically over variable applications $z \ e_1 \dots e_n$.

$$\boxed{\begin{array}{c} Q \mid \Gamma \vdash e \overset{\text{red}}{\vdash} \tau \\ \downarrow \quad \uparrow \quad \uparrow \quad \uparrow \\ \text{out} \quad \text{in} \quad \text{in} \quad \text{in} \end{array}}, \boxed{\begin{array}{c} Q \mid \Gamma \vdash e \overset{\text{blue}}{\vdash} \tau \\ \downarrow \quad \uparrow \quad \uparrow \quad \downarrow \\ \text{out} \quad \text{in} \quad \text{in} \quad \text{out} \end{array}}, \boxed{\begin{array}{c} Q \mid \Gamma \vdash t \overset{\text{blue}}{\vdash} \rho \\ \downarrow \quad \uparrow \quad \uparrow \quad \downarrow \\ \text{out} \quad \text{in} \quad \text{in} \quad \text{out} \end{array}}, \boxed{\begin{array}{c} Q \mid \Gamma \vdash_{\text{GEN}} t \overset{\text{blue}}{\vdash} \sigma \\ \downarrow \quad \uparrow \quad \uparrow \quad \downarrow \\ \text{out} \quad \text{in} \quad \text{in} \quad \text{out} \end{array}} \text{ with } \models Q$$

We use $\overset{\text{red}}{\vdash} \tau$ for checking type τ , and $\overset{\text{blue}}{\vdash} \tau$ for inferring type τ .

$$\begin{array}{c} \frac{\exists! e. Q \mid \Gamma \Vdash z \overset{\text{red}}{\vdash} \tau \rightsquigarrow e}{Q \mid \Gamma \vdash z \overset{\text{red}}{\vdash} \tau} \text{VAR} \qquad \frac{Q_1 \mid \Gamma \vdash e \overset{\text{red}}{\vdash} \tau_1 \quad Q_2 \vdash \tau_1 \approx \tau_2}{Q_1, Q_2 \mid \Gamma \vdash (e : \tau_1) \overset{\text{red}}{\vdash} \tau_2} \text{ANN} \\[10pt] \frac{Q \mid \Gamma, z : \tau_1 \vdash e \overset{\text{red}}{\vdash} \tau_2}{Q \mid \Gamma \vdash \lambda z. e \overset{\text{red}}{\vdash} \tau_1 \rightarrow \tau_2} \text{FUN} \qquad \frac{Q_1 \mid \Gamma \vdash \lambda z. e \overset{\text{red}}{\vdash} \alpha_1 \rightarrow \alpha_2 \quad Q_2 \vdash \alpha_1 \rightarrow \alpha_2 \approx \alpha \text{ fresh } \alpha_1, \alpha_2}{Q_1, Q_2 \mid \Gamma \vdash \lambda z. e \overset{\text{red}}{\vdash} \alpha} \text{IFUN} \\[10pt] \frac{Q \mid \Gamma \vdash e \overset{\text{red}}{\vdash} \alpha \text{ fresh } \alpha}{Q \mid \Gamma \vdash e \overset{\text{blue}}{\vdash} \alpha} \text{INF} \qquad \frac{Q \mid \Gamma, z : \alpha \vdash t \overset{\text{blue}}{\vdash} \rho \text{ fresh } \alpha}{Q \mid \Gamma \vdash \lambda ?z. t \overset{\text{blue}}{\vdash} ?\hat{z} : \alpha \rightarrow \rho} \text{IMPLICIT} \\[10pt] \frac{Q_1 \mid \Gamma \vdash_{\text{GEN}} t \overset{\text{blue}}{\vdash} \sigma \quad Q_2 \mid \Gamma, z : \sigma \vdash e \overset{\text{red}}{\vdash} \tau}{Q_1, Q_2 \mid \Gamma \vdash \text{let } z = t \text{ in } e \overset{\text{red}}{\vdash} \tau} \text{LET} \qquad \frac{Q_0 \mid \Gamma \vdash t \overset{\text{blue}}{\vdash} \rho \quad (Q, \sigma) = \text{gen}(Q_0, \Gamma, \rho)}{Q \mid \Gamma \vdash_{\text{GEN}} t \overset{\text{blue}}{\vdash} \sigma} \text{GEN} \\[10pt] \frac{Q_1 \mid \Gamma \vdash e_1 \overset{\text{red}}{\vdash} \alpha \rightarrow \tau \quad Q_2 \mid \Gamma \vdash e_2 \overset{\text{red}}{\vdash} Q_1[\alpha] \text{ fresh } \alpha}{Q_1, Q_2 \mid \Gamma \vdash e_1 e_2 \overset{\text{red}}{\vdash} \tau} \text{APP-ARG} \\[10pt] \frac{\begin{array}{l} \text{least } i \text{ with } 0 \leq i \leq n \text{ fresh } \alpha_{i+1}, \dots, \alpha_n \\ Q_1 \mid \Gamma \vdash e_1 \overset{\text{blue}}{\vdash} \tau_1 \quad \dots \quad Q_i \mid \Gamma \vdash e_i \overset{\text{blue}}{\vdash} \tau_i \quad Q = Q_1, \dots, Q_i \\ Q_0 \mid \Gamma \vdash z \overset{\text{red}}{\vdash} Q[\tau_1] \rightarrow \dots \rightarrow Q[\tau_i] \rightarrow \alpha_{i+1} \rightarrow \dots \rightarrow \alpha_n \rightarrow \tau \\ Q_{i+1} \mid \Gamma \vdash e_{i+1} \overset{\text{red}}{\vdash} Q_0[\alpha_{i+1}] \quad \dots \quad Q_n \mid \Gamma \vdash e_n \overset{\text{red}}{\vdash} Q_0[\alpha_n] \end{array}}{Q_0, Q_1, \dots, Q_n \mid \Gamma \vdash z e_1 \dots e_i \dots e_n \overset{\text{red}}{\vdash} \tau} \text{APP-VAR} \end{array}$$

$$\begin{array}{ll} \text{gen} : (Q, \Gamma, \sigma) \rightarrow (Q, \sigma) \\ \text{gen}(Q \cdot \alpha = \tau, \Gamma, \rho) &= \text{gen}(Q, \Gamma, [\alpha := \tau] \rho) \quad \text{if } \alpha \notin \text{ftv}(Q, \Gamma) \\ \text{gen}(Q, \Gamma, \sigma) &= \text{gen}(Q, \Gamma, \forall \alpha. \sigma) \quad \text{if } \alpha \notin \text{ftv}(Q, \Gamma) \wedge \alpha \in \text{ftv}(\sigma) \\ \text{gen}(Q, \Gamma, \sigma) &= (Q, \sigma) \quad \text{if } (\text{dom}(Q) \cup \text{ftv}(\sigma)) \subseteq \text{ftv}(\Gamma) \end{array}$$

Fig. 5. Syntax-directed bidirectional type checking rules for static overloading with syntactic implicit parameters. We always prefer [APP-VAR] over [APP-ARGC] when applicable.

The [APP-VAR] rule looks a bit intimidating but essentially it just tries to infer the least amount of arguments i such that we can disambiguate z , and then propagates the remaining argument types into the remaining argument expressions. This strategy is straightforward to implement: first try to disambiguate z (without any inference of the arguments) and keep inferring one argument at a time until z can be disambiguated, and eventually use checking rules for the remaining arguments. Even though the rule has the drawback of a left-to-right bias, it seems to work well in practice with the Koka language (but as remarked by Leijen and Ye [2025] further refinements are possible).

These type rules are all fully determined by the syntax (and the shape of the propagated type in [FUN]/[IFUN]). Moreover, the rules are carefully constructed such that the (result) prefix Q only contains constraints induced by the structure of the program and types: at all leaf nodes in the derivation Q is empty, except for [EQ-VAR]. As a consequence, any well-typed program has only a

single possible derivation:

Theorem 2.3. (*Principal type derivations*)

For any derivations $Q \mid \Gamma \vdash_{\text{GEN}} e \dot{\vdash} \sigma \rightsquigarrow e_1$ and $Q' \mid \Gamma \vdash_{\text{GEN}} e \dot{\vdash} \sigma' \rightsquigarrow e_2$, we have $Q = Q'$, $\sigma = \sigma'$, and $e_1 = e_2$.

See the proof in Appendix B.3 of the supplement. This theorem is important as it also implies that the elaboration is deterministic with coherent semantics.

2.9 Scope Based Disambiguation

The current rules do not take the scope of a qualified name into account. Sometimes that can be inconvenient as we would like to be able to override definitions in an outer scope. For example,

```
fun secret-list()
  fun myint/show(i:int) = "*"
  show([1,2])
```

where we would like to elaborate to `list/show([1,2],myint/show)`. However, with the current rules this example would be rejected as both `int/show` and `myint/show` match. In such cases though, one might expect instead to prefer the definition that originated from an inner scope.

We can formalize this by changing the `[VAR]` rule to instead prefer a minimal solution instead of unique solution. Suppose we annotate every variable z with its scope depth i as z^i . The global scope depth is 0 and increments inside every let bound definition. We write $e \preceq e'$ if an elaborated expression e is either equal to e' , or resolved to an inner scope relative to e' . We define this as:

$$z^i e_1 \dots e_n \preceq z'^j e'_1 \dots e'_k = i > j \vee (z = z' \wedge e_1 \preceq e'_1 \wedge \dots \wedge e_n \preceq e'_n)$$

(where we note that if $z = z'$ then $n = k$ for elaborated expressions). Let's write the condition on the current `[VAR]` as $P(Q, e) = Q \mid \Gamma \Vdash z \dot{\vdash} \tau \rightsquigarrow e$. We can then then rephrase `[VAR]` as:

$$\frac{\exists! e. P(Q, e) \wedge (\forall Q' e'. P(Q', e') \Rightarrow e \preceq e')}{Q \mid \Gamma \vdash z \dot{\vdash} \tau} \text{VAR-SCOPE}$$

Our Koka implementation uses this rule instead of `[VAR]` to prioritize qualified definitions in an inner scope.

2.10 Coherence and Stability

Schrijvers et al. [2019] present the Cochis calculus of coherent implicits. In this paper they argue that systems that support implicit programming should be *coherent* and *stable*.

Coherence is defined as a valid program always having exactly one meaning. Our system is coherent: this follows from the `[VAR]` rule as it requires a unique elaboration in combination with Theorem 2.3 which guarantees unique derivations. As an example of coherence, Schrijvers *et al* present a classic example from Haskell as `show (read "3")`. Such expressions are rejected in Haskell as the type class resolution is ambiguous. For example, we could use instances for `Int` or `Bool` and many others to resolve the `read`. Similarly, assuming appropriate qualified `read` definitions, our system would reject this since the type context cannot resolve `read` unambiguously. Unlike Haskell though, we can disambiguate the example explicitly without needing a type signature, for example as `show (int/read "3")`. Since overloading in our system is just elaboration to fully qualified names we can always disambiguate manually in our system without needing extra type annotations.

Schrijvers et al. [2019] also argue that implicit type systems should be *stable*, where instantiation of type variables does not affect resolution. In essence, this means that using an overloaded variable at a more specific type does not change the semantics. This is not always the case for systems that would disambiguate based on a most specific type for example. We can show stability in our system formally over the variable resolution rules:

Theorem 2.4. (*Stability*)

If $Q_1 \mid \Gamma \vdash z \dot{\vdash} \tau \rightsquigarrow e$ with $\models (\theta, Q_1)$, then we also have $Q_2 \mid \theta\Gamma \vdash z \dot{\vdash} \theta\tau \rightsquigarrow e$ (with $Q_1 \sqsubseteq (\theta, Q_2)$).

See the proof in Appendix B.4 of the supplement. However, in the Cochis system the notion of stability is stronger than this. They show the following example in Haskell where overlapping instances are allowed:

```
class Trans α where trans :: α → α
instance Trans α where trans x = x
instance Trans Int where trans x = x + 1
```

with the definition

```
bad :: ∀α. α → α
bad x = trans x
```

Due to the type signature, the type class can be resolved unambiguously to the generic one. However, that also means that *bad* 1 evaluates to 1, while *trans* 1 evaluates to 2, and it is argued this breaks equational reasoning. We do not necessarily agree with this: as we saw before, a type annotation is essential to the Haskell semantics (as it does not have a dynamic untyped semantics), and we cannot just leave it out and replace *bad* with *trans* directly. In our system there is a similar situation where we can only do equational reasoning on fully qualified identifiers. Suppose we define:

```
generic/trans x = x    and    int/trans x = x + 1
```

then the expression:

```
bad = λx. trans x
```

would be rejected as it is ambiguous. We need to either qualify the *trans* variable, as for example:

```
generic/bad = λx. generic/trans x    or    int/bad = λx. int/trans x
```

or propagate *trans* explicitly as an implicit parameter:

```
bad = λ?trans. λx. trans x
```

In the last case, *bad* 1 indeed equals *trans* 1 (as it gets elaborated to *bad int/trans* 1).

But what about the original example in Cochis with the polymorphic type signature:

```
bad : ∀α. α → α = λx. trans x
```

In this case, we can argue there are two acceptable solutions. The first design (I) is that *trans* should be resolved to *generic/trans* since the type of the argument is an abstract α and thus only the *generic/trans* can match. The other argument (II), as made in Cochis, is that this example should be rejected as the instantiation of $\forall\alpha. \alpha \rightarrow \alpha$ with *int* would make this example ambiguous again (i.e. it is not stable under type application).

Cochis uses a separate stable predicate to check for this form of ambiguity, but it turns out that we can modularly describe either design in our system as two variants of the type scheme checking rule. First we extend the syntax to allow type scheme annotations for σ (with $\text{ftv}(\sigma) \subseteq \text{ftv}(\Gamma)$) as let $z : \sigma = t$ in e :

$$\frac{Q_1 \mid \Gamma \vdash t \dot{\vdash} \sigma \quad Q_2 \mid \Gamma, z : \sigma \vdash e \dot{\vdash} \tau}{Q_1, Q_2 \mid \Gamma \vdash \text{let } z : \sigma = t \text{ in } e \dot{\vdash} \tau} \text{LET-ANN}$$

For checking mono-types, we already have the [ANN] rule. For ρ types, we extend this now to check implicit parameters:

$$\frac{\hat{z} = x \quad Q \mid \Gamma, z : \tau \vdash t \dot{\vdash} \rho}{Q \mid \Gamma \vdash \lambda?z.t \dot{\vdash} ?x:\tau \rightarrow \rho} \text{ANN-IMPLICIT}$$

This leaves us with checking polymorphic quantifiers $\forall\alpha$. Usually, such checking rules are expressed algorithmically by replacing the polymorphic type variables with fresh type constants c (also called *skolem constants* [Peyton Jones et al. 2007]):

$$\frac{Q_1 \mid \Gamma \vdash t \dot{\vdash} [\bar{\alpha} := \bar{c}] \rho \quad (Q_2, ()) = \text{gen}(Q_1, \Gamma, ()) \quad \bar{c} \notin Q_2 \quad \text{fresh } \bar{c}}{Q_2 \mid \Gamma \vdash t \dot{\vdash} \forall \bar{\alpha}. \rho} \text{ANN-SKOLEM}$$

The condition $\bar{c} \notin Q_2$ ensures that the fresh type constants do not escape the scope of their binding. We need to use gen here to remove any unused constraints mentioning $\bar{c}$ (which can occur due to [INST-QUANTIFY]). Consider for example:

$$\frac{\frac{id : \forall \alpha. \alpha \rightarrow \alpha \in \Gamma \quad \frac{\{\alpha=c\} \mid \Gamma \vdash \alpha \rightarrow \alpha \approx c \rightarrow c \quad \text{fresh } \alpha}{\{\alpha=c\} \mid \Gamma \vdash \forall \alpha. \alpha \rightarrow \alpha \sqsubseteq c \rightarrow c \rightsquigarrow \cdot} \text{VAR}}{\exists! id. \{\alpha=c\} \mid \Gamma \Vdash id \dot{\vdash} c \rightarrow c \rightsquigarrow id} \text{VAR} \\ \frac{\{\alpha=c\} \mid \Gamma \vdash id \dot{\vdash} c \rightarrow c \quad (\emptyset, _) = \text{gen}(\{\alpha=c\}, \Gamma, ()) \quad \bar{c} \notin \emptyset}{\emptyset \mid \Gamma \vdash id \dot{\vdash} \forall \alpha. \alpha \rightarrow \alpha} \text{VAR}$$

This follows design (I) where our example gets resolved to *generic/trans* as the integer alternative cannot match with the abstract skolem constant (i.e. $\not\vdash c \approx \text{int}$). This is the rule that is used in the Koka implementation.

However, we can also implement design (II) by not using skolem constants in the first place, and instead checking *afterwards* whether a type variable was indeed used polymorphically [Leijen 2008]:

$$\frac{Q_1 \mid \Gamma, \bar{\alpha} \vdash t \dot{\vdash} \rho \quad (Q_2, \forall \bar{\alpha}. \rho) = \text{gen}(Q_1, \Gamma, \rho) \quad \text{fresh } \bar{\alpha}}{Q_2 \mid \Gamma \vdash t \dot{\vdash} \forall \bar{\alpha}. \rho} \text{ANN-SCHEME}$$

Here, we instantiate the polymorphic type as regular fresh type variables, but check afterwards that these did not unify with any type, nor escaped the scope through any other type constraints. To prevent generalizing early over the $\bar{\alpha}$ binders, we extend Γ to include type variable bindings as well as $\Gamma, \bar{\alpha}$ (which can be seen as a shorthand for a sequence of anonymous bindings as $\Gamma, _ : \bar{\alpha}$).

Interestingly, for standard Hindley-Milner either approach is valid and it makes no difference. In our case though, by keeping $\bar{\alpha}$ as regular type variables, higher up in the derivation there can now be multiple matches for overloads, and we may no longer satisfy the unique $\exists!e$ condition in the [VAR] rule. In particular, for the *trans* example, both *generic/trans* and *int/trans* now match (since $\{\alpha := \text{int}\} \vdash \alpha \approx \text{int}$), and the example is rejected due to ambiguity – corresponding to the second design (as advocated by Cochis).

In Cochis, the stability property is shown by translating from the core calculus into System-F and showing that static reduction of type application preserves typing. For our case this strategy does not quite work since once we elaborate to fully qualified names we already have an untyped dynamic semantics. We can show stability more directly though over the [ANN-SCHEME] rule:

Theorem 2.5. (Polymorphic Stability)

If $Q_1 \mid \Gamma \vdash t \dot{\vdash} \forall \bar{\alpha}. \rho \rightsquigarrow e$ (using [ANN-SCHEME]), we also have $Q_2 \mid \Gamma \vdash t \dot{\vdash} [\bar{\alpha} := \bar{\tau}] \rho \rightsquigarrow e$ (for any $\bar{\tau}$ with $\text{ftv}(\bar{\tau}) \subseteq \text{ftv}(\Gamma)$).

See Appendix B.4 of the supplement for the proof.

2.11 Definition Stability

There is another form of stability that is quite important in practice, which we call definition stability. We call a system *definition stable* if the semantics of an existing function does not change

silently when adding a new definition. Consider for example the following Haskell program:

```
class Trans  $\alpha$  where trans :  $\alpha \rightarrow \alpha$ 
instance Trans  $\alpha$  where trans x = x
foo = trans 1
```

where $foo = 1$. However, suppose we enable overlapping instances and add a more specific definition (maybe in an imported module):

```
instance Trans Int where trans i = i + 1
```

In this case, the *trans* is resolved now to the most specific instance and *foo* is now 2 (and thus Haskell with overlapping instances is not definition stable). Dually, Scala prefers the most general instance for implicits and is also not definition stable. In our system, adding either definition always results in an ambiguity error (and is therefore definition stable). Even with the extension to scope based disambiguation (Section 2.9) our system is still definition stable as any outer definition is never preferred to an inner one.

3 TYPE INFERENCE

We essentially extended the standard type rules with just two new rules for implicit parameter instantiation ([INST-IMPLICIT]) and variable overloading ([VAR-QUALIFY]). We saw that even though each rule is simple, their recursive nature makes the rules quite expressive. Perhaps even *too* expressive since the potential recursion between these rules can easily lead to infinite derivations! Consider for example a function *foo* that requires itself as an implicit parameter, as *foo*: $?foo:(int \rightarrow int) \rightarrow int \rightarrow int$. This is a problem for type inference as trying to derive an application like *foo* 1 would lead to infinite recursion.

3.1 Undecidability of Type Checking

As it is, general type checking for our current rules turns out to be undecidable! In particular, we can show it is possible to encode a Turing machine on the type level, where type checking becomes equivalent to showing termination of the encoded Turing machine. See Appendix A of the supplement. for an example of encoding of a 3-state busy-beaver Turing machine [Rado 1962] in Koka. In particular, we can create types for the symbols 0 and 1, and the tape of the machine as:

```
type sym0          type cons<a,b> // tape with head a and tail b
type sym1          type inf       // infinite tape of zeros
```

Furthermore, we can create machine states and a full Turing machine configuration as:

```
type state-a        type state-b        type state-c
abstract type config<s,l,r> = Start // the machine config: <state, left tape, right tape>
val start : config<state-a,inf,inf> = Start // starting machine configuration with all zeros
```

We can then encode the machine state transitions in the types. For example, for the 3-state busy-beaver, if we are in state A with the head being 0, we write 1 to the head, shift the tape to the right, and continue in state B:

```
fun ab/transition( st : config<state-a,cons<x,l>,cons<sym0,r>>,
                  ?transition : (config<state-b,l,cons<x,cons<sym1,r>>>) -> () ) : () = ()
```

We write a overloaded function for each transition, as well as a halting rule (in state C):

```
fun cend/transition( st : config<state-c,l,cons<sym1,r>> ) : () = ()
```

See the full example in Appendix A of the supplement where we also show how can expand the *inf* tape automatically to *cons<sym0,inf>* when so required. We can then start the machine as *transition (start)*, where static overloading and implicit parameters elaborate the plain *transition* to a full sequence of all state transitions – for the 3-state busy-beaver machine this elaborates to 14 state transitions.

$$\begin{array}{c}
\boxed{Q \mid \Gamma \Vdash_n z : \tau \rightsquigarrow e} \quad \boxed{Q \mid \Gamma \vdash_n \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}} \\
\begin{array}{cccccc}
\downarrow & \uparrow & \uparrow & \uparrow & \uparrow & \downarrow \\
\text{out} & \text{in} & \text{in} & \text{in} & \text{in} & \text{out}
\end{array}
\end{array}$$

$$\frac{}{\emptyset \mid \Gamma \Vdash_0 z : \tau \rightsquigarrow \perp} \text{VAR-CUT} \quad \frac{z : \sigma \in \Gamma \quad Q \mid \Gamma \vdash_n \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}}{Q \mid \Gamma \Vdash_{n+1} z : \tau \rightsquigarrow z \bar{e}} \text{VAR-DIRECTK}$$

$$\frac{z \notin \text{dom}(\Gamma) \quad m/z : \sigma \in \Gamma \quad Q \mid \Gamma \vdash_n \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}}{Q \mid \Gamma \Vdash_{n+1} z : \tau \rightsquigarrow m/z \bar{e}} \text{VAR-QUALIFYK}$$

$$\frac{(\exists! e. Q \mid \Gamma \vdash_K z : \tau \rightsquigarrow e) \quad \perp \notin e}{Q \mid \Gamma \vdash z : \tau} \text{VAR-K}$$

Fig. 6. Type rules with finite derivations (up to depth K). Instantiation passes n unchanged.

3.2 Recovering Decidability

In order to make the system decidable we need to impose some decreasing measure on the derivations to ensure termination of the type checking algorithm. One way to do this is to simply limit the depth of the derivation tree ending in a $[\text{VAR}]$ to be of at most K deep for some fixed K .

Figure 6 gives a new set of rules for variable derivations (and instantiation) as $Q \mid \Gamma \Vdash_n z : \tau$ where we pass a decreasing bound $n \in \mathbb{N}$. Each application of $[\text{VAR-DIRECTK}]$ or $[\text{VAR-QUALIFYK}]$ decreases n (while n is passed unchanged through instantiation). Both $[\text{VAR-DIRECTK}]$ or $[\text{VAR-QUALIFYK}]$ can now only be applied for $n > 0$. In the case $n = 0$, the $[\text{VAR-CUT}]$ rule applies which always elaborates to an “infinite” expression $\perp$. The new $[\text{VAR-K}]$ rule is now extended to always reject any solutions that contain $\perp$ expressions.

These rules are clearly terminating since n strictly decreases. As such they are incomplete with respect to the original rules since some derivations are now rejected as too deep. We do have soundness though: any valid derivation using $[\text{VAR-K}]$ is also valid in the original rules:

Theorem 3.6. (*Soundness of the Finite Rules*)

If $Q \mid \Gamma \vdash z : \tau \rightsquigarrow e$ with $[\text{VAR-K}]$, then also $Q \mid \Gamma \vdash z : \tau \rightsquigarrow e$ with $[\text{VAR}]$.

See the proof in Appendix B.2 of the supplement. It turns out that the soundness property is a bit subtle. In particular, in an earlier design we left out the rule $[\text{VAR-CUT}]$ as it seems we can just not have any valid derivation that requires $n = 0$. This would be unsound with respect to the original rules though!

Currently, any potential infinite derivation has a solution containing $\perp$ using $[\text{VAR-CUT}]$. As such, it might be that the $\exists! e$ condition in the $[\text{VAR-K}]$ rule is now not satisfied as valid finite solutions compete with such “infinite” solutions (which is also why the $\perp \notin e$ condition is outside the scope of the uniqueness condition). If we leave out the $[\text{VAR-CUT}]$ rule though, there would be no valid derivation instead for such “infinite” ones, and in that case a particular finite solution may suddenly be unique and accepted. In such case though, the original rules may still find a finite derivation (of depth $> K$) for our “infinite” one and reject it (as it becomes ambiguous). Although we are the first to formalize this, this issue with soundness was also remarked by White et al. [2015].

3.3 Maintaining History

Of course, our formalization based on a fixed depth K of $[\text{VAR-DIRECTK}]$ or $[\text{VAR-QUALIFYK}]$ applications is clear but perhaps also a bit naive in practice. However, we can see that in principle any

decreasing measure would suffice to limit the depth of the derivation tree.

As our approach is name based, we observe that for any infinite recursion to happen, it must be that we try to resolve the same qualified name more than once. Instead of a fixed K , we can instead pass a *history* H of each name $m/z : \tau$ in $[\text{VAR-QUALIFYK}]$ (and $z : \tau$ in $[\text{VAR-DIRECTK}]$). One way to ensure termination is to simply reject any derivation that leads to a repeated name z in the history. That would be too restrictive though as it would reject for example *show* $\llbracket 1 \rrbracket$ (where *list/show* is resolved twice).

A better way is to only allow a derivation for $z : \tau$ to proceed if for the last $z : \tau'$ appearing in the history, we have that τ is “smaller” than τ' . We can define smaller here for example as the count of constructors in the type. This allows our earlier *show* $\llbracket 1 \rrbracket$ to be accepted again.

In the Koka implementation, we refine this a bit further where we require for $z : \tau$ that there are either fewer than N entries for z in the history, and otherwise that the τ is smaller than any one of the last N entries $z : \tau'$ in the history (where $N = 4$ in our case). This allows for some limited recursion where the size of the type stays can the same (or be larger) for some iterations. For example, our implementation can now accept the 3-state busy beaver problem we discussed earlier (but of course, still not arbitrary Turing machines).

3.4 Implementing Type Inference

As remarked by Leijen and Ye [2025], making the type rules syntax directed and under a prefix, we can essentially “read off” the inference algorithm. This make the formalism particularly useful not only for users to have a precise specification, but also for compiler implementors to ensure it is implemented accordingly. As shown by Leijen and Ye [2025], we can represent valid prefixes in a canonical form as a regular substitution, and implement unification and composition of two prefixes as $\text{unify}(\tau_1, \tau_2) : Q$ and $\text{compose}(Q_1, Q_2) : Q$ (see Appendix C of the supplement). These call an operation $\text{fail}()$ of an effect handler [Leijen 2021] if no unification exists or when the union of the two prefixes does not have a valid solution. We can now directly implement the $Q \mid \Gamma \vdash_n \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ rules for instantiation as:

$$\begin{aligned} \text{inst} &: (\Gamma, \mathbb{N}, \tau, \tau) \rightarrow (Q, \bar{e}) \\ \text{inst}(\Gamma, n, \tau_1, \tau_2) &= (\text{unify}(\tau_1, \tau_2), \cdot) \\ \text{inst}(\Gamma, n, \forall \alpha. \sigma, \tau) &= \text{inst}(\Gamma, n, \sigma[\alpha := \text{fresh}()], \tau) \\ \text{inst}(\Gamma, n, ?x : \tau_1 \rightarrow \rho, \tau_2) &= \text{val } (Q_1, \bar{e}) = \text{inst}(\Gamma, n, \rho, \tau_2); (Q_2, e) = \text{resolve}(\Gamma, n, x, Q[\tau_1]) \\ &\quad \text{in } (\text{compose}(Q_1, Q_2), e \bar{e}) \end{aligned}$$

Here we assume another operation *fresh* to generate fresh type variables. Each definition corresponds immediately to the instantiation rules $[\text{INST-MONO}]$, $[\text{INST-QUANTIFY}]$, and $[\text{INST-IMPLICIT}]$. That leaves the *resolve* function to implement the $Q \mid \Gamma \Vdash_n z : \tau \rightsquigarrow e$ variable rules:

$$\begin{aligned} \text{resolve} &: (\Gamma, \mathbb{N}, z, \tau) \rightarrow (Q, e) \\ \text{resolve}(\Gamma, 0, z, \tau) &= \text{infinite}() \\ \text{resolve}(\Gamma, n, z, \tau) \mid z : \sigma \in \Gamma &= \text{val } (Q, \bar{e}) = \text{inst}(\Gamma, n-1, \sigma, \tau) \text{ in } (Q, z \bar{e}) \\ \text{resolve}(\Gamma, n, z, \tau) &= \text{val } (m/z, \sigma) = \text{forallq}(\Gamma, z); (Q, \bar{e}) = \text{inst}(\Gamma, n-1, \sigma, \tau) \text{ in } (Q, m/z \bar{e}) \end{aligned}$$

Again, each definition corresponds to the variable rules $[\text{VAR-CUT}]$, $[\text{VAR-DIRECTK}]$, and $[\text{VAR-QUALIFYK}]$. However, we see two new operations here: *infinite* and *forallq*. The call to *infinite*() returns a bottom expression as $(\emptyset, \perp)$, but *forallq*(Γ, z) is special as it should return a list of *all* $m/z : \sigma \in \Gamma$. Here is where we can use an effect handler to neatly express this in a modular way while keeping the implementations of *inst* and *resolve* as close as possible to the type rules. In particular, we can

implement *resolve* under a handler that implements a list of successes [Philip Wadler 1985]:

```

explore : (Γ, z, τ) → [(Q, e)]
explore(Γ, z, τ) = handle resolve(Γ, K, z, τ)
  return x      → [x]
  fail()        → []
  infinite()    → resume((∅, ⊥))
  forallq(Γ, z) → [x | m/z : σ ∈ Γ, x ← resume(m/z, σ)]

```

Normal results (return x) are wrapped in a singleton list $[x]$, while failures return an empty list. The *infinite*() operation just resumes at the call site with a bottom expression (as $(\emptyset, \perp)$). The *forallq*(Γ, z) operation is the most interesting one as it resumes for each $m/z : \sigma \in \Gamma$ and appends all returned solutions. In the type checking algorithm *check*, we can now call *explore* to implement the variable case as:

```

check : (Γ, e, τ) → (Q, e)
...
check(Γ, z, τ) = match explore(Γ, z, τ)
  [(Q, e)] | ⊥ ∉ e → (Q, e)      unique and not ⊥
  []          → fail()          unresolved
  _          → fail()          ambiguous or infinite

```

This implements the $\exists!e$ condition by essentially exploring all possible derivations and returning normally only if there is a single successful derivation. The connection between using a list of successes and the $\exists!e$ condition may not be immediately apparent, but we can show that the algorithm is sound and complete with respect to our finite type rules:

Theorem 3.7. (*Algorithmic Soundness*)

If $\text{check}(\Gamma, z, \tau) = (Q, e)$, then also $Q \mid \Gamma \vdash z \vdash \tau \rightsquigarrow e$ (with $[\text{VAR-K}]$).

Theorem 3.8. (*Algorithmic Completeness*)

If $Q \mid \Gamma \vdash z \vdash \tau \rightsquigarrow e$ (with $[\text{VAR-K}]$), then also $\text{check}(\Gamma, z, \tau) = (Q, e)$.

See the proofs in Appendix B.1 of the supplement.

3.5 Optimizing Exploration

Even though the previous theorems show our algorithm is correct, it is not the most efficient implementation. We can make various improvements to the *explore* function to optimize the exploration of the search space.

First, we observe that we never actually look at the individual solutions if there is more than one solution. As such, we only consider a valid singleton solution $[(Q, e)]$, an empty solution $[]$, or multiple solutions. We can represent this more efficiently using a specialized data type:

```
type sol α = One(α) | None | Amb
```

Furthermore, we also observe that we never accept any infinite solution with $\perp \in e$. Let's only use *One*((Q, e)) if $\perp \notin e$, and directly use *Amb* for infinite solutions as well. We can then define *check* as:

```

check(Γ, z, τ) = match exploreopt(Γ, z, τ)
  One(x) → x
  _      → fail()

```

We do not need to change the definitions of *resolve* or *inst* as these use abstract operations – instead we only need to modify the handler to use our new representation:

```

exploreopt( $\Gamma, z, \tau$ ) = handle resolve( $\Gamma, K, z, \tau$ )
  return  $x \rightarrow \text{One}(x)$ 
  fail()    $\rightarrow \text{None}$ 
  infinite()  $\rightarrow \text{Amb}$ 
  ...

```

Here we optimized the infinite operation: instead of resuming we observe that any evaluation with *infinite()* will eventually have an *Amb* result, and thus we can directly return with an *Amb* result without resuming. The main advantage of our new representation is in the definition of *forallq* as we can now cut the evaluation short as soon as we encounter any *Amb* result without needing to explore all possible derivations:

```

forallq( $\Gamma, z$ )  $\rightarrow$ 
  let find(current, candidates) = match candidates
    Nil  $\rightarrow$  current
    Cons( $m/z : \sigma, \text{rest}$ )  $\rightarrow$  match resume( $m/z, \sigma$ )
      Amb    $\rightarrow$  Amb
      None   $\rightarrow$  find(current, rest)
      One( $x$ )  $\rightarrow$  match current
        One( $y$ )  $\rightarrow$  Amb
        None   $\rightarrow$  find(One( $x$ ), rest)
  in find(None,  $\forall m\sigma. m/z : \sigma \in \Gamma$ )

```

The *find(current, candidates)* function iterates through all candidates using the *current* result which is either *None* or *One*((Q, e)). As soon as we find an *Amb* result, we can immediately return with *Amb* without trying more solutions. Similarly when already have a valid current solution, and we find another valid solution, we can immediately return with *Amb* as well.

In our experience, cutting the exploration short as soon an ambiguous or infinite derivation is found is quite important in practice. In particular, the [APP-VAR] rule often requires backtracking to get enough type information from the first argument expressions – failing fast is important here. As shown in Section 2.9, we can also prefer inner scope definitions instead of requiring unique solutions. This can actually be used to optimize the exploration even further. In particular, we can directly skip trying resolving any m/z from an outer scope as soon as we found a solution for a definition in an inner scope.

4 SYNTACTIC IMPLICITS AND STATIC OVERLOADING IN PRACTICE

The system described in this paper is fully implemented in the Koka language [Leijen 2019 2021], and it is quite heavily used in the standard library for functions like *show*, (*==*), *map*, *comparison*, etc. In Koka, application is not curried and all arguments are in between parenthesis. In contrast to our calculus, the implicit arguments always come last which fits better with the uncurried syntax. In particular, the programmer can now pass implicit parameters explicitly by adding them as regular parameters. For example, we can write *show*([1], *fn*(x) " $*$ ") to display a list of integers where all elements are shown as $*$. In the case of multiple implicit parameters, we can pass a subset of them by name as well, *show*((1, *True*), ?*snd*/show=*fn*(x) " $*$ ") for example.

As described in Section 2.9, Koka prefers inner-scope definitions, and as discussed in Section 3.3, instead of cutting the search off at an arbitrary depth our implementation maintains a history where the size of the types of implicit parameters must decrease within 4 recursive elaborations. This makes it possible to still accept the 3-state busy beaver program (see Appendix A of the supplement).

A nice advantage of our system is that there is no need to declare overloaded functions in advance, we just have regular function definitions. This also puts no constraints on the type of these functions. This allows us in particular to overload tuple selectors for example:

```
fun tuple/fst ( x : (a,b) ) : a
fun triple/fst( x : (a,b,c) ) : a
```

and more generally selectors for fields in any datatype. This is not possible with type classes [Odersky et al. 1995; P. Wadler and Blott 1989] as a specific type signature needs to be declared upfront.

4.1 Default Implementations

In the standard library, we have various definitions to compare basic types, like:

```
type order = Lt | Eq | Gt
fun bool/cmp( x : bool, y : bool ) : order
fun int/cmp( x : int, y : int ) : order
fun list/cmp( xs : list<a>, ys : list<a>, ?cmp : a -> order ) : order
```

Given these definitions, we can now write a generic equality operator:

```
fun default/(==)( x : a, y : a, ?cmp : (a,a) -> order ) : bool
  match cmp(x,y)
  | Eq -> True
  | _  -> False
fun (!=)( x : a, y : a, ?(==) : (a,a) -> bool ) : bool
  !=(==)(x,y)
```

This way, for any new data type, we only need to define the comparison function `cmp`, and get (in)equality for free. However, there is a problem with this as well. In particular, we may have a new data type that can determine equality more efficiently without doing a full comparison. This already happens in the standard library where equality between integers has a fast primitive:

```
fun int/(==)( x : int, y : int ) : bool
  prim-int-eq(x,y)
```

Unfortunately, since these definitions overlap, an expression like `1==2` will always be rejected as both `int/(==)` and `default/(==)` could apply. Haskell type classes address this problem by allowing default implementations in class definitions that are used unless an instance provides an explicit implementation. In our case, one way around this is to name `default/(==)` differently, and at each new data type require the user to write an explicit `(==)` definition that can either use the default one based on `cmp` or be specialized for that data type.

In Koka we use a different approach though: we pretend that any definition in the `default/` namespace is defined in a scope outside the global module scope, i.e. at scope depth `-1` instead of `0`. This means that when there is a choice between a regular definition and a default one, the regular one is preferred due to the scope disambiguation (Section 2.9). Our `1==2` example is now unambiguous, and elaborated to `int/(==)(1,2)`.

Unfortunately, the addition of `default/` can cause definition instability (Section 2.11). We may have a function that uses a variable that is resolved to a `default/` definition. If we add a more specific version, that one is now considered to be in an inner scope relative to `default/`, and is preferred without an ambiguity error. This is not ideal and we would like extend the compiler to at least warn in such cases (while preserving the convenience of the outer `default/` scope).

4.2 Grouping Operations

Oftentimes, we need multiple operations that logically belong together. It can be cumbersome to pass each one individually as an implicit parameter in general. Haskell type classes naturally group operations together where for example the `Num` class has addition, multiplication, etc. No such thing exists in our system but instead we can pass structures directly as an implicit parameter.

For example, we can define:

```
struct num<a>
  (+): (a, a) -> a
  (*): (a, a) -> a
```

and provide some “instances” as:

```
val int/num : num<int> = Num( (+), (*) )
```

However, using such structures can be a bit cumbersome, for example:

```
fun fadd( x : a, y : a, z : a, ?num : num<a> ) : a
  match num
    Num((+),(*)) -> x + (y * z)
```

Fortunately, Koka provides dot notation to automatically unpack structures, and we can write:

```
fun fadd( x : a, y : a, z : a, .?num : num<a> ) : a
  x + (y * z)
```

which is essentially syntactic sugar for the previous explicit version. The dot notation also recursively unpacks any base members which allows for linear hierarchies, for example for `monad<m>` and `monadplus<m>`. This simple mechanism can work surprisingly well, but of course it is limited as well compared to more specialized mechanisms like type classes. We hope to gain more experience in practice to evaluate its limits.

4.3 Phantom Implicit

The value passed at the call site for a syntactic implicit parameters, is just the plain name of the parameter. If the name is not in scope at the call site the program is rejected. However, we can imagine having a special set of implicit parameter names (and associated types) for which the compiler can supply a more elaborate argument term. We call such special names *phantom implicits*. For example, our Koka implementation defines the special implicit parameter names `kk-line : int` and `kk-file : string` (in `std/core/debug`). If in `[INST-IMPLICIT]` these names cannot be resolved, the compiler can instead elaborate to specific value: in these cases the current source file line number and name respectively. For example:

```
fun assert-line( condition : bool, msg : string, ?kk-line : int ) : exn ()
  if condition then () else throw("at " ++ kk-line.show ++ ": " ++ msg)
```

and then use it as `assert-line(False, "failed")`. Since `kk-line` is not defined at the call site, the compiler instead provides the current source line number for the implicit parameter, and elaborate to `assert-line(False, "failed", 42)`. Note that we can still define `kk-line` explicitly. Unlike a pre-processor macro, we can *abstract* over these special names. For example, we can define:

```
fun assert-fline(condition : bool, msg : string, ?kk-line : int, ?kk-file : string ) : exn ()
  assert-line(condition, kk-file ++ ": " ++ msg)
```

Here, `assert-line` is elaborated to `assert-line(condition, kk-file ++ ": " ++ msg, kk-line)`. Since `kk-line` is in scope the compiler does not need to provide another value. However, when `assert-fline` is used where `kk-line` and `kk-file` are not in scope, the current line number and file of the call site are provided implicitly again.

For both the `kk-line` and `kk-file` phantom implicits, the compiler can supply their argument value just based on their name. However, we can also imagine supplying the argument value of a phantom implicit based on the caller’s type context. This would lead to a design that is more similar to how type classes get resolved: we can imagine letting the user define type classes and instances – when a “type class” phantom implicit needs to be resolved, for example `num : num<int>`, the instance declarations can be used to construct the correct dictionary to pass.

4.3.1 Divergence. Koka has at the moment a single phantom implicit that is resolved based on its type, namely `hdiv : hdiv<h,a,e>`. The type `hdiv` is an abstract type and users can never create

a value of this type. Only the compiler is allowed to do so if the type constraints are satisfied. In particular, the type `hdiv<h,a,e>` signifies that if the type `a` can contain a reference to the heap `h`, then the effect `e` must contain the divergence effect (`div`). This is used in Koka to correctly infer potential divergence where one stores self referential functions in a mutable heap. Leijen [2014] gives the following example of Landin’s knot:

```
fun landin() : div ()           // div is inferred!
  val r = ref(fn() ())         // initialize with a parameter-less function that returns unit
  fun self()
    (!r)()                     // self calls the function stored in r
  r := self                    // store self in r now
  self()                       // and call it
```

Here we create a reference `r` in the global heap, and later store function `self` in there, where `self` calls the function stored in `r`. Even though the function has no syntactic recursion, it diverges and Koka needs to infer the `div` effect in the type of `landin`. In order to detect such sneaky divergence, we define the dereference operation (`!`) with the `hdiv` phantom implicit:

```
fun ref/(!)( r : ref<h,a>, ?hdiv : hdiv<h,a,e> ) : <read<h>|e> a
```

This reads a value of type `a` from a reference `r` in heap `h`, and has a `read<h>` effect and possibly more effects `e`. By including the phantom implicit here, we make explicit that the `e` effect now must contain the divergence effect `div` as well whenever the value type `a` can contain the same heap `h` in its type, i.e. whenever it is self referential. In the `landin` example above, the name `hdiv` for the phantom implicit is not bound so the compiler tries to supply a value instead, with the type `hdiv<h, () -> <read<h>|e> (), e>`. Since `h` is clearly in the value type `() -> <read<h>|e>`, the compiler unifies `e` with `<div|_>` to satisfy the constraint on `hdiv` implicits, and then supplies an internal dummy value as the evidence for the `hdiv` phantom implicit. This ensures that the `landin` function indeed gets the `div` effect.

In most cases in practice, the value type is a simple type like `int` and it is clear that it does not contain `h`. In such cases, the compiler directly supplies an internal dummy value again as evidence, but now without unifying `e` with the divergence effect. Sometimes though, the value type is polymorphic and at the call site it may still be undetermined if `h` can be in `a`. Just like with the previous `kk-line` example, we can then further abstract over the `hdiv` implicit to delay its evaluation – the user may not be able to create `hdiv` evidence values, but they are able to pass them around:

```
fun read2( r : ref<h,a>, ?hdiv : hdiv<h,a,e> ) : <read<h>|e> (a,a)
  val x = !r in (x,x)
```

In this example, the polymorphic dereference `!` is elaborated to `ref/(!)(r,hdiv)`, where we essentially defer resolving the `hdiv` evidence to the call site of `read2` instead.

5 RELATED WORK

Since the main idea of syntactic implicit parameters is so straightforward, it is not easy to directly compare against more sophisticated systems like type classes or Scala implicits. Instead, we try to highlight how particular usage scenarios are addressed in various systems.

Type classes [P. Wadler and Blott 1989] are a very elegant form of implicits that has seen widespread success in languages like Haskell and Lean for example. An important difference with our system is that type class constraints are automatically generalized if they cannot be resolved locally, essentially introducing an implicit parameter for the dictionary at runtime. For example, `parens = λx. "(" ++ show x ++ ")"` would result in the type `parens: Show α ⇒ α → String` in Haskell. In contrast, in our system this definition is rejected, and we need to explicitly state that we are abstracting over `show`, as `parens = λ?show. λx. "(" ++ show x ++ ")"` (or qualify `show` manually to disambiguate). Kovács [2020] investigate how much to generalize the types of inferred dictionary parameters in dependently typed languages.

Another difference is that type classes require global uniqueness of instances to be coherent. As a consequence, instances live in another scope than regular values. Moreover, once we define say an instance for *Num Int*, we cannot locally override this with another instance to do, say, modular arithmetic; Sozeau and Oury [2008] and Dreyer et al. [2007] investigate how such first-class type classes could look like. Kiselyov and Shan [2004] show how to use phantom types and rank-2 polymorphism with type classes to add implicit configuration modularly in a program and obtain coherence without global uniqueness.

Scala [Odersky 2016; Odersky et al. 2017] is one of the first languages to use implicits extensively. Like type classes, the implicits have a separate scope, and are resolved by their type. For example:

```
implicit val number : Int = 1
def add(x : Int)(implicit y : Int) = x + y
add(2)
```

where the `add(2)` would be elaborated to `add(2, number)` as that is the only `Int` in the implicit scope. Scala users usually declare new nominal types to avoid ambiguities. B. C. Oliveira et al. [2010] show how Scala implicits can be used to express type classes as well. However, just like in our system, one still needs to explicitly declare the required implicit parameters. Both type classes and Scala style implicits resolve implicits by their type. B. C. d. S. Oliveira et al. [2012] and later Schrijvers et al. [2019] develop an implicit calculus that expresses the essence of these systems. The latter paper in particular studies the coherence and stability properties.

In contrast, a system that uses explicit names to resolve implicit parameters is presented by Lewis et al. [2000] (and later by Jones [1999]). This design of implicit parameters is based on type classes where using an implicit name `?x` gives rise to an implicit constraint. For example, let $f = ?y + 2$ in $(f \text{ with } ?y = 1)$ evaluates to 3. The `?y` here is an implicit parameter, and, just like type classes, the type of f is generalized to $?y : \text{Int} \Rightarrow \text{Int}$. The `with` construct is used to resolve such constraints with a particular value. Since names like `?y` do not live in the usual lexical scope but are instead type level constraints, this may lead to surprises. For example, consider the expression $(\text{let } f = ?y + 2 \text{ in } f + (f \text{ with } ?y = 1))$ with `?y = 2` which evaluates to 7 (and not 8) even though lexically it may appear as if the outer binding for `?y` binds the occurrence in f .

White et al. [2015] describe an extension to the OCaml language for ad-hoc polymorphism inspired by Scala implicits and modular type classes. Their modular implicits are based on type-directed implicit module parameters, and elaborate straightforwardly into OCaml’s first-class functors. Devriese and Piessens [2011] present *instance arguments* for the Agda language. These are inspired by both Scala’s implicits and Agda’s existing implicit arguments.

Because we resolve implicits by name, we can use history (Section 3.3) to carefully avoid non-termination of the search. In type based systems this can be more challenging. Agda’s search avoids any recursion in the first place at the cost of needing to be more explicit. Rocq [2025 Rocq Prover 2025], has a configurable recursion limit (`Typeclasses Depth`), which defaults to infinite depth-first search. Schrijvers et al. [2019], provides a termination property based on the type head (implicit constraints), which ensures that the size of the type is decreasing. White et al. [2015] provides a termination property similar to ours, where successive instantiations of the same implicit functor must be decreasing in the size of collected constraints.

6 CONCLUSION

We presented a formal system for syntactic implicit parameters and static overloading. Even though each feature is rather straightforward, their interaction turned out to be surprisingly expressive. With the implementation in Koka we hope to gain more experience in using this at scale and extend it in interesting ways. In particular, we would like to improve on grouping (Section 4.2), and study the application of phantom implicits in future work.

REFERENCES

- Devriese, and Piessens. 2011. On the Bright Side of Type Classes: Instance Arguments in Agda. *ACM SIGPLAN Notices* 46 (9). ACM New York, NY, USA: 143–155.
- Dreyer, Harper, Chakravarty, and Keller. 2007. Modular Type Classes. In *Proceedings of the 34th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 63–70.
- Jones. 1999. *Exploring the Design Space for Type-Based Implicit Parameterization*. Technical report, Oregon Graduate Institute.
- Kiselyov, and Shan. 2004. Functional Pearl: Implicit Configurations—or, Type Classes Reflect the Values of Types. In *Proceedings of the 2004 ACM SIGPLAN Workshop on Haskell*, 33–44.
- Kovács. 2020. Elaboration with First-Class Implicit Function Types. *Proceedings of the ACM on Programming Languages* 4 (ICFP). ACM New York, NY, USA: 1–29.
- Leijen. Sep. 2008. HMF: Simple Type Inference for First-Class Polymorphism. In *Proceedings of the 13th ACM Symposium of the International Conference on Functional Programming*. ICFP’08. Victoria, Canada. doi:<https://doi.org/10.1145/1411204.1411245>.
- Leijen. 2014. Koka: Programming with Row Polymorphic Effect Types. In *MSFP’14, 5th Workshop on Mathematically Structured Functional Programming*. doi:<https://doi.org/10.4204/EPTCS.153.8>.
- Leijen. 2019. Koka Repository. <https://github.com/koka-lang/koka>.
- Leijen. 2021. The Koka Language. <https://koka-lang.github.io>.
- Leijen, and Ye. Sep. 2024. *Principal Type Inference under a Prefix: A Fresh Look at Static Overloading (TR)*. MSR-TR-2024-34. Microsoft Research.
- Leijen, and Ye. Jun. 2025. Principal Type Inference under a Prefix: A Fresh Look at Static Overloading. *Proceedings of ACM Programming Languages (PLDI)* 9 (PLDI). ACM. doi:<https://doi.org/10.1145/3729308>.
- Lewis, Launchbury, Meijer, and Shields. 2000. Implicit Parameters: Dynamic Scoping with Static Types. In *Proceedings of the 27th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 108–118. POPL’00. ACM, Boston, MA, USA.
- Odersky. Dec. 2016. Implicit Function Types. <https://www.scala-lang.org/blog/2016/12/07/implicit-function-types.html>. Blog post.
- Odersky, Blanvillain, Liu, Biboudis, Miller, and Stucki. 2017. Simplicity: Foundations and Applications of Implicit Function Types. *Proceedings of the ACM on Programming Languages* 2 (POPL). ACM New York, NY, USA: 1–29.
- Odersky, Wadler, and Wehr. 1995. A Second Look at Overloading. In *Proceedings of the Seventh International Conference on Functional Programming Languages and Computer Architecture*, 135–146.
- Bruno C.d.S. Oliveira, Moors, and Odersky. 2010. Type Classes as Objects and Implicits. *ACM Sigplan Notices* 45 (10). ACM New York, NY, USA: 341–360.
- Bruno C.d.S. Oliveira, Schrijvers, Choi, Lee, and Yi. 2012. The Implicit Calculus: A New Foundation for Generic Programming. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*, 35–44. PLDI ’12. ACM, Beijing, China.
- Peyton Jones, Vytiniotis, Weirich, and Shields. 2007. Practical Type Inference for Arbitrary-Rank Types. *Journal of Functional Programming* 17 (1): 1–82. doi:<https://doi.org/10.1017/S0956796806006034>.
- Pierce. Feb. 2002. *Types and Programming Languages (TAPL)*. 1st edition. The MIT Press, Cambridge, Massachusetts 02142.
- Rado. 1962. On Non-Computable Functions. *Bell System Technical Journal* 41 (3). Wiley Online Library: 877–884.
- Rocq Prover. 2025. <https://rocq-prover.org/>. Accessed: 2025-11-13.
- Schrijvers, Oliveira, Wadler, and Marntirosian. 2019. COCHIS: Stable and Coherent Implicits. *Journal of Functional Programming*. {bib-url}.
- Selsam, Ullrich, and Moura. 2020. Tabled Typeclass Resolution. arXiv:[cs.PL/2001.04301](https://arxiv.org/abs/cs.PL/2001.04301).
- Sozeau, and Oury. 2008. First-Class Type Classes. In *International Conference on Theorem Proving in Higher Order Logics*, 278–293. Springer.
- P. Wadler, and Blott. 1989. How to Make Ad-Hoc Polymorphism Less Ad Hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 60–76. POPL’89. ACM, Austin, Texas, USA. doi:<https://doi.org/10.1145/75277.75283>.
- Philip Wadler. 1985. How to Replace Failure by a List of Successes a Method for Exception Handling, Backtracking, and Pattern Matching in Lazy Functional Languages. In *Functional Programming Languages and Computer Architecture*, edited by Jean-Pierre Jouannaud, 113–128. Springer Berlin Heidelberg, Berlin, Heidelberg.
- White, Bour, and Yallop. 2015. Modular Implicits. In *Proceedings ML Family/OCaml Users and Developers Workshops, Gothenburg, Sweden, September 4-5, 2014*, edited by Oleg Kiselyov and Jacques Garrigue, 198:22–63. Electronic Proceedings in Theoretical Computer Science. Open Publishing Association. doi:<https://doi.org/10.4204/EPTCS.198.2>.

A TURING ENCODING OF A 3-STATE BUSY BEAVER

This example encodes a 3-state “busy beaver” Turing machine program by Ivars Peterson¹ in the type system. It has the following transition table:

current symbol	state A			state B			state C		
	write	move	next	write	move	next	write	move	next
0	1	R	B	1	L	A	1	L	B
1	1	L	C	1	R	B	1	N	HALT

We encode the symbols, tape, and states using abstract types. The tape is represented as a cons list in the type language using phantom type parameters. The tape is encoded finitely with type `inf` representing an infinite sequence of `sym0`.

```
type sym0
type sym1

// a tape
type cons<a,b>    // tape with head 'a' and tail 'b'
type inf

// all states also match on 'inf' and produce 'cons<sym0,inf>' to simulate infinite tape
type state-a
type state-b
type state-c
```

We encode the configuration as the current state of the finite state machine and the left and right tape using phantom type parameters in the `config` type. The machine can read or write the first symbol on the right tape (the head).

```
// the machine configuration: <state, left tape, right tape>
abstract type config<s,a,c>
  Start
```

We give an explicit type annotation to initialize the phantom types for the starting machine configuration.

```
// starting machine configuration with all zeros
val start : config<state-a,inf,cons<sym0,inf>> = Start
```

Transitions are encoded using functions with implicit `?trans` constraints with the final `halt` transition requiring no further implicits.

```
// halting state
fun halt/trans() : ()
  ()
```

When `state == A`, and `head == 0` we encode the transition to write `1`, shift the tape right and continue in state `B` as follows:

```
// stateA: if head==0 then write 1, shift the tape right, and continue in state B
fun a/b/trans( st : config<state-a,cons<x,l>,cons<sym0,r>>,
  ?trans : (config<state-b,l,cons<x,cons<sym1,r>>>) -> () ) : ()
  ()
```

Similar transitions are encoded for the other states:

¹https://en.wikipedia.org/wiki/Turing_machine_examples#3-state_Busy_Beaver

```

// stateA: if head==1 then write 1, shift the tape left, and continue in state C
fun a/c/trans( st : config<state-a,l,cons<sym1,cons<x,r>>>,
              ?trans : (config<state-c,cons<sym1,l>,cons<x,r>>>) -> () ) : ()
()

// stateB: if head==0 then write 1, shift the tape left, and continue in state A
fun b/a/trans( st : config<state-b,l,cons<sym0,cons<x,r>>>,
              ?trans : (config<state-a,cons<sym1,l>,cons<x,r>>>) -> () ) : ()
()

// stateB: if head==1 then write 1, shift the tape right, and continue in state B
fun b/b/trans( st : config<state-b,cons<x,l>,cons<sym1,r>>>,
              ?trans : (config<state-b,l,cons<x,cons<sym1,r>>>) -> () ) : ()
()

// stateC: if head==0 then write 1, shift the tape left, and continue in state B
fun c/b/trans( st : config<state-c,l,cons<sym0,cons<x,r>>>,
              ?trans : (config<state-b,cons<sym1,l>,cons<x,r>>>) -> () ) : ()
()

// stateC: if head==1 then write 1, do not move, and continue in state HALT
fun c/end/trans( st : config<state-c,l,cons<sym1,r>>>,
                ?trans : () -> () ) : ()
()

```

Because the tape encoding is finite, at times we must expand the tape by turning the infinite portion into zeros. However, we cannot do so arbitrarily since that can lead to ambiguity. To ensure uniqueness, we only expand the side of the tape that needs expansion in order for a transition to happen. For example the `a/b/trans` shifts from the left tape to the right, so we need a corresponding function that if given state-a and right hand tape with head `sym0` expands `inf`. This is done in the `inf/a/b/trans` rule below, which then allows the original `a/b/trans` to proceed.

```

fun inf/a/b/trans( st : config<state-a,inf,cons<sym0,r>>>,
                  ?trans : (config<state-a,cons<sym0,inf>,cons<sym0,r>>>) -> () ) : ()
()
fun inf/a/c/trans( st : config<state-a,l,cons<sym1,inf>>>,
                  ?trans : (config<state-a,l,cons<sym1,cons<sym0,inf>>>) -> () ) : ()
()
fun inf/b/b/trans( st : config<state-b,inf,cons<sym1,r>>>,
                  ?trans : (config<state-b,cons<sym0,inf>,cons<sym1,r>>>) -> () ) : ()
()
fun inf/b/a/trans( st : config<state-b,l,cons<sym0,inf>>>,
                  ?trans : (config<state-b,l,cons<sym0,cons<sym0,inf>>>) -> () ) : ()
()
fun inf/c/b/trans( st : config<state-c,l,cons<sym0,inf>>>,
                  ?trans : (config<state-c,l,cons<sym0,cons<sym0,inf>>>) -> () ) : ()
()

```

We can evaluate the program by using the following `main` function and the command: `koka -e busy-beaver.kk`.

```

fun main()
  trans( start )
  println("done.")

```

In VSCode we can hover over the call which shows us the inferred implicit chain representing the transitions. We show the resulting chain below where we have aligned the `inf` and corresponding non-`inf` rules for readability.

```

// expands to a 14 state expansion (with the Koka termination check disabled),
// and matches the execution of:
// <https://en.wikipedia.org/wiki/Turing_machine_examples#3-state_Busy_Beaver>
/*
inf/a/b/trans(?trans=a/b/trans(,
  b/a/trans(,
  inf/a/c/trans(,a/c/trans(,
  inf/c/b/trans(,c/b/trans(,
  inf/b/a/trans(,b/a/trans(,
  a/b/trans(,
  b/b/trans(,
  b/b/trans(,
  b/b/trans(,
  inf/b/b/trans(,b/b/trans(,
  b/a/trans(,
  a/c/trans(,
  c/end/trans(,
  halt/trans))))))))))
*/

```

To simplify the handling of infinite expansion, we can recognize that only a single function for shifting left / shifting right is needed. Since infinite tapes are always expanded to add a `sym0` we need to know that there exists a rule that given the current head, state, and a `sym0` where there currently is `inf`, will shift that `sym0` off of that side. However, this requires us to not only know the shape of the head of the transition rule, but also the tail.

With higher-order implicit parameters (parameters whose types contain implicit parameters), we can expect an implicit parameter `?trans` that shifts to the right (matching on the shape of both the head and tail of the transition), and also require the same implicit matching only the head of the transition rule, requiring further implicits to be handled by the caller.

```

// Expand the 'inf' tape if the state would transition from the infinite side if it had a sym0.
fun inf/l/trans( st : config<s,inf,cons<h,r>>,
  ?a/trans : (config<s,cons<sym0,inf>,cons<h,r>>,
    ?trans: (config<s1,inf,r2>) -> () ) -> (),
  ?b/trans: (config<s,cons<sym0,inf>,cons<h,r>>) -> () ) : ()
()
// Expand the 'inf' tape if the state would transition from the infinite side if it had a sym0.
fun inf-r/trans( st : config<s,l,cons<h,inf>>,
  ?a/trans : (config<s,l,cons<h,cons<sym0,inf>>>,
    ?trans: (config<s1,l2,cons<h2,inf>>) -> () ) -> (),
  ?b/trans: (config<s,l,cons<h,cons<sym0,inf>>>) -> () ) : ()
()

```

B PROOFS

B.1 Algorithmic Soundness and Completeness

To prove that the algorithm is sound and complete, we first inline the handler in *explore* to make the list of successes explicit without using operations.

$$\begin{aligned} \text{resolve}(\Gamma, 0, z, \tau) &= [(\emptyset, \perp)] \\ \text{resolve}(\Gamma, n, z, \tau) \mid z : \sigma \in \Gamma &= [(Q, z \bar{e}) \mid (Q, \bar{e}) \leftarrow \text{inst}(\Gamma, n-1, \sigma, \tau)] \\ \text{resolve}(\Gamma, n, z, \tau) &= [(Q, m/z \bar{e}) \mid m/z : \sigma \in \Gamma, (Q, \bar{e}) \leftarrow \text{inst}(\Gamma, n-1, \sigma, \tau)] \end{aligned}$$

$$\begin{aligned} \text{inst}(\Gamma, n, \tau_1, \tau_2) &= [(Q, \cdot) \mid Q \leftarrow \text{unify}(\tau_1, \tau_2)] \\ \text{inst}(\Gamma, n, \forall \alpha. \sigma, \tau) &= \text{inst}(\Gamma, n, \sigma[\alpha := \text{fresh}()], \tau) \\ \text{inst}(\Gamma, n, ?x : \tau_1 \rightarrow \rho, \tau_2) &= \\ &[(Q, e \bar{e}) \mid (Q_1, \bar{e}) \leftarrow \text{inst}(\Gamma, n, \rho, \tau_2), (Q_2, e) \leftarrow \text{resolve}(\Gamma, n, x, Q_1[\tau_1]), Q \leftarrow \text{compose}(Q_1, Q_2)] \end{aligned}$$

Then *check* directly calls *resolve*, and does a case analysis on the list of results.

$$\begin{aligned} \text{check} : (\Gamma, e, \tau) &\rightarrow (Q, e) \\ \dots \\ \text{check}(\Gamma, z, \tau) &= \text{match } \text{resolve}(\Gamma, K, z, \tau) \\ &[(Q, e)] \mid \perp \notin e \rightarrow (Q, e) \quad \text{unique and not } \perp \\ [] &\rightarrow \text{fail}() \quad \text{unresolved} \\ - &\rightarrow \text{fail}() \quad \text{ambiguous or infinite} \end{aligned}$$

For soundness and completeness, we will need a uniqueness Lemma stating that the rules and algorithm only produce corresponding unique results.

Lemma B.9. (*Uniqueness*)

$\exists! e. Q \mid \Gamma \Vdash_n z : \tau \rightsquigarrow e$ iff $\text{resolve}(\Gamma, n, z, \tau) = [(Q, e)]$.

If *resolve* were not complete, it could produce one solution instead of two, which could return a value from *check* when there should be none. If it were not sound, it could produce one solution instead of zero, which could return a value from *check* when the implicit is not derivable using the rules. So before proving the uniqueness lemma and soundness and completeness of *check* we first need to prove soundness and completeness of *resolve*.

Because *resolve* and *inst* are mutually recursive, we need to prove their soundness and completeness together by mutual induction. To ensure completeness and termination for *resolve* we can use stepwise induction on n , since n always decreases. However, instantiation does not affect n , and we can simply prove by induction on the derivation given the soundness and completeness of *resolve* at step n , this is well-founded because each inductive step of *inst* makes the type scheme smaller.

For the purposes of these proofs we consider $\Vdash_0$ as $\Vdash_n$ with a premise $n = 0$ and $\Vdash_{n+1}$ as $\Vdash_n$ with premises $n = n' + 1 \wedge n > 0 \wedge n' = n - 1$ and n replaced by n' within the rule. This allows for cleaner proofs where the conclusions are all in terms of n , and the inductive case can be in terms of $n - 1$, matching the algorithm.

Additionally, we generalize the proofs over all solutions $S = [(Q, e)]$ (*resolve*) and $Ss = [(Q, \bar{e})]$ (*instantiation*) satisfying the inference rules, since the algorithm works via list comprehension over all solutions.

Lemma B.10. (*Resolution Soundness*)

If $\text{resolve}(\Gamma, n, z, \tau) = S$ at step n and *inst* is sound (1) at step $n - 1$ where $n > 0$, then also

$$\forall (Q, e) \in S. Q \mid \Gamma \Vdash_n z : \tau \rightsquigarrow e.$$

Proof. Proceeding by cases on n first with $n = 0$.

Case We have $\text{resolve}(\Gamma, 0, z, \tau) = [(\emptyset, \perp)]$ where $n = 0$, and need to show that $(\emptyset, \perp)$ is derivable in the rules. We can derive $(\emptyset, \perp)$ via [VAR-CUT] since $n = 0$.

Case When $n > 0$ (2), we have two subcases.

subcase $\text{resolve}(\Gamma, n, z, \tau) \mid z : \sigma \in \Gamma$ (4) = $[(Q, z \bar{e}) \mid (Q, \bar{e}) \leftarrow \text{inst}(\Gamma, n-1, \sigma, \tau)]$ (3). By soundness of inst (1), we have for each element in (3), $Q \mid \Gamma \vdash_{n-1} \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ (5). And by [VAR-DIRECTK] and (2,5), for each $(Q, z \bar{e})$ we have $Q \mid \Gamma \Vdash_n z : \tau \rightsquigarrow z \bar{e}$.

subcase $\text{resolve}(\Gamma, n, z, \tau) = [(Q, m/z \bar{e}) \mid m/z : \sigma \in \Gamma, (Q, \bar{e}) \leftarrow \text{inst}(\Gamma, n-1, \sigma, \tau)]$. From the algorithm we have $(Q, \bar{e}) \leftarrow \text{inst}(\Gamma, n-1, \sigma, \tau)$ (3), and $m/z : \sigma \in \Gamma$ (4) with $z \notin \Gamma$ (5) (implicitly by falling through the match). We can derive $Q \mid \Gamma \vdash_{n-1} \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ (6) by soundness of inst (1,3). For each $(Q, m/z \bar{e})$ we can show $Q \mid \Gamma \Vdash_n z : \tau \rightsquigarrow m/z \bar{e}$ by [VAR-QUALIFYK] using (4,5,6).

Lemma B.11. (*Instantiation Soundness*)

If $\text{inst}(\Gamma, n, \tau_1, \tau_2) = Ss$ at step n and resolve is sound (1) at step n then also

$\forall (Q, \bar{e}) \in Ss. Q \mid \Gamma \vdash_n \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$.

Proof. We prove by induction on the derivation.

Case The first case applies to mono types: $\text{inst}(\Gamma, n, \tau_1, \tau_2) = [(Q, \cdot) \mid Q \leftarrow \text{unify}(\tau_1, \tau_2)]$. We derive $Q \vdash \tau_1 \approx \tau_2$ (2) by the soundness of unify (Lemma C.28). We then can show $Q \mid \Gamma \vdash_n \tau_1 \sqsubseteq \tau_2 \rightsquigarrow \cdot$ for all $(Q, \bar{e})$ in the resulting list by [INST-MONO] and (2).

Case This case applies to type schemes with quantifiers: $\text{inst}(\Gamma, n, \forall \alpha. \sigma, \tau) = \text{inst}(\Gamma, n, \sigma[\alpha := \text{fresh}()], \tau)$. Follows directly the inductive hypothesis (given that fresh is correct).

Case The last case applies to implicits types:

$\text{inst}(\Gamma, n, ?x : \tau_1 \rightarrow \rho, \tau_2) =$

$[(Q, e \bar{e}) \mid (Q_1, \bar{e}) \leftarrow \text{inst}(\Gamma, n, \rho, \tau_2), (Q_2, e) \leftarrow \text{resolve}(\Gamma, n, x, Q_1[\tau_1]), Q \leftarrow \text{compose}(Q_1, Q_2)]$

From the algorithm we have $(Q_1, \bar{e}) \leftarrow \text{inst}(\Gamma, n, \rho, \tau_2)$ (3), $(Q_2, e) \leftarrow \text{resolve}(\Gamma, n, x, Q_1[\tau_1])$ (4), and $Q \leftarrow \text{compose}(Q_1, Q_2)$ (5). By induction (2,3), we have $Q_1 \mid \Gamma \vdash_n \rho \sqsubseteq \tau_2 \rightsquigarrow \bar{e}$ (6). By soundness of resolve (1,4), we have $Q_2 \mid \Gamma \Vdash_n x : Q_1[\tau_1] \rightsquigarrow e$ (7). We can derive from the soundness of compose (Lemma C.29) and (5), that we have $\models (Q_1, Q_2)$ (8) We can apply [INST-IMPLICIT] now by (6,7,8).

We also need completeness of resolve and inst .

Lemma B.12. (*Resolution Completeness*)

If $Q \mid \Gamma \Vdash_n z : \tau \rightsquigarrow \bar{e}$ at step n and inst is complete at step $n-1$ where $n \geq 1$ then

$\text{resolve}(\Gamma, n, z, \tau) = S \wedge (Q, e) \in S$.

Proof. We proceed by case analysis of the derivation. Because the rules have mutually exclusive premises we can consider each case with respect to a single line of the algorithm.

Case From [VAR-CUT] we have $\emptyset \mid \Gamma \vdash_0 z : \tau \rightsquigarrow \perp$ and $n = 0$ which requires us to include $(\emptyset, \perp)$ in the result when $n = 0$: $\text{resolve}(\Gamma, 0, z, \tau) = [(\emptyset, \perp)]$. This is mutually exclusive with the other cases where the conclusions require $n > 0$ which contradicts $n = 0$.

Case From [VAR-DIRECTK] we have $z : \sigma \in \Gamma$ (2), and $Q \mid \Gamma \vdash_{n-1} \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ (3). The corresponding line in the algorithm is mutually exclusive with the other cases since the conclusion only applies for $n > 0$, and ensures (2). From the inductive hypothesis and (3) we have $\text{inst}(\Gamma, n-1, \sigma, \tau) = Ss \wedge (Q, \bar{e}) \in Ss$ (4). From (4) we are required to include $(Q, z \bar{e})$ in the results in this case:

$\text{resolve}(\Gamma, n, z, \tau) \mid z : \sigma \in \Gamma = [(Q, z \bar{e}) \mid (Q, \bar{e}) \leftarrow \text{inst}(\Gamma, n-1, \sigma, \tau)]$

Case From [VAR-QUALIFYK] we have that $z \notin \text{dom}(\Gamma)$ (2) $m/z : \sigma \in \Gamma$ (3) and $Q \mid \Gamma \vdash_{n-1} \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ (4) with $n > 0$ (5). We do not match the first (5) or second (2) cases of the algorithm making this mutually exclusive. From the inductive hypothesis and (4), we have $\text{inst}(\Gamma, n-1, \sigma, \tau) = Ss \wedge (Q, \bar{e}) \in Ss$

(6). From (6) we must include $(Q, m/z \bar{e})$ in the results:

$$\text{resolve}(\Gamma, n, z, \tau) = [(Q, m/z \bar{e}) \mid m/z : \sigma \in \Gamma, (Q, \bar{e}) \leftarrow \text{inst}(\Gamma, n-1, \sigma, \tau)].$$

Lemma B.13. (*Instantiation Completeness*)

If $Q \mid \Gamma \vdash_n \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ at step n and resolve (1) is complete at step n where $n \geq 0$ then also $\text{inst}(\Gamma, n, \tau_1, \tau_2) = Ss \wedge (Q, \bar{e}) \in Ss$.

Proof. We proceed by induction on the derivation tree.

Case From $Q \mid \Gamma \vdash_n \tau_1 \sqsubseteq \tau_2 \rightsquigarrow \cdot$ we have $Q \vdash \tau_1 \approx \tau_2$ and by the completeness of unification, we are required to include $(Q, \cdot)$ in our result: $\text{inst}(\Gamma, n, \tau_1, \tau_2) = [(Q, \cdot) \mid Q \leftarrow \text{unify}(\tau_1, \tau_2)]$.

Case From $Q \mid \Gamma \vdash_n \forall \alpha. \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ we have $Q \mid \Gamma \vdash_n \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ (3) and fresh α . By the induction hypothesis and (3) we have that $Q \mid \Gamma \vdash_n \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ requires us to include results from $\text{inst}(\Gamma, n, \sigma[\alpha := \text{fresh}()], \tau)$, given fresh type variables.

Case From $Q_1, Q_2 \mid \Gamma \vdash_n ?x:\tau_1 \rightarrow \rho \sqsubseteq \tau_2 \rightsquigarrow e \bar{e}$ we have $Q_1 \mid \Gamma \vdash_n \rho \sqsubseteq \tau_2 \rightsquigarrow \bar{e}$ (3) and $Q_2 \mid \Gamma \Vdash_n x : Q_1[\tau_1] \rightsquigarrow e$ (4). By the induction hypothesis and (3) we have that we must consider each $(Q_1, \bar{e}) \leftarrow \text{inst}(\Gamma, n, \rho, \tau_2)$ and for each of those we derive $(Q_2, e) \leftarrow \text{resolve}(\Gamma, n, x, Q_1[\tau_1])$ via the induction hypothesis and (4). Finally due to the completeness of compose we have that (Q_1, Q_2) exists, requiring us to include $(Q, e \bar{e})$ in the results for this case:

$$\begin{aligned} \text{inst}(\Gamma, n, ?x:\tau_1 \rightarrow \rho, \tau_2) = \\ [(Q, e \bar{e}) \mid (Q_1, \bar{e}) \leftarrow \text{inst}(\Gamma, n, \rho, \tau_2), (Q_2, e) \leftarrow \text{resolve}(\Gamma, n, x, Q_1[\tau_1]), Q \leftarrow \text{compose}(Q_1, Q_2)] \end{aligned}$$

Finally we need that uniqueness holds for both directions.

Proof. (Of Lemma B.9 (Uniqueness)) As a reminder, to prove soundness, we must first prove that unique derivations correspond to unique resolutions in the algorithm:

$$\exists! e \ Q \mid \Gamma \Vdash_n z : \tau \rightsquigarrow \bar{e} \text{ iff } \text{resolve}(\Gamma, n, z, \tau) = [(Q, e)].$$

Case In the forwards direction we have $Q \mid \Gamma \Vdash_n z : \tau \rightsquigarrow \bar{e}$ (1) and $\exists! e$ (2). From completeness and (1) we have that $\text{resolve}(\Gamma, n, z, \tau) = S$ where $(Q, e) \in S$. Assume there are multiple (Q, e) in S : that immediately contradicts (2). So we only have one result, and can show $\text{resolve}(\Gamma, n, z, \tau) = [(Q, e)]$

Case In the backwards direction we have $\text{resolve}(\Gamma, n, z, \tau) = [(Q, e)]$ (1). By soundness and (1) we have that $Q \mid \Gamma \Vdash_n z : \tau \rightsquigarrow \bar{e}$ (2). Furthermore, assume that there is some other $Q' \mid \Gamma \Vdash_n z : \tau \rightsquigarrow \bar{e}'$, then via completeness and (1) we have a contradiction. Therefore the solution (2) is unique $\exists! e$ (3). With (2) and (3) we are able to show the conclusion.

$$\text{check} : (\Gamma, e, \tau) \rightarrow (Q, e)$$

$$\begin{aligned} \dots \\ \text{check}(\Gamma, z, \tau) &= \text{match } \text{resolve}(\Gamma, K, z, \tau) \\ &[(Q, e)] \mid \perp \notin e \rightarrow (Q, e) \\ [] &\rightarrow \text{fail()} && \text{unresolved} \\ - &\rightarrow \text{fail()} && \text{ambiguous or infinite} \end{aligned}$$

Proof. (Of Theorem 3.7 (Algorithmic Soundness)) We must prove that given $\text{check}(\Gamma, z, \tau) = (Q, e)$, then also

$Q \mid \Gamma \vdash z : \tau \rightsquigarrow e$ (1) (with [VAR-K]). The only case where check produces a result is when resolve returns a unique result e with an associated Q (2). We can show (1) via the uniqueness lemma and (2).

Proof. (Of Theorem 3.8 (Algorithmic Completeness)) We must prove that given $Q \mid \Gamma \vdash z \dot{\vdash} \tau \rightsquigarrow e$ also $\text{check}(\Gamma, z, \tau) = (Q, e)$ (1). The rule only produces a result if $\Vdash_n$ has a unique result e with an associated Q (2). We can show (1) via the uniqueness lemma and (2).

B.2 Soundness of the Finite Type Rules

Proof. (Of Theorem 3.6) We must show that given $Q \mid \Gamma \vdash z \dot{\vdash} \tau \rightsquigarrow e$ (1) with $[\text{VAR-K}]$, then $Q \mid \Gamma \vdash z \dot{\vdash} \tau \rightsquigarrow e$ with $[\text{VAR}]$.

This reduces trivially to showing that if $Q \mid \Gamma \Vdash_n z \dot{\vdash} \tau \rightsquigarrow e$ (1), $\exists! e$ (2), and $\perp \notin e$ (3) then $\exists! e, Q \mid \Gamma \Vdash z \dot{\vdash} \tau \rightsquigarrow e$ (4).

We prove by cases on the number and kinds of solutions from the finite rules $\Vdash_n$.

Case None or multiple solutions: Contradiction that we have a solution (1) and that it is unique (2).

Case One solution:

subcase $\perp \in e$: Contradiction that the solution is not infinite (3)

subcase $\perp \notin e$: Straightforward proof by induction on the derivation, the rules are identical except for passing n . We can proceed by case analysis on the derivation of $\perp \notin e$ in the inductive cases, which allows us to eliminate rule $[\text{VAR-CUT}]$ which is the only other difference.

More detailed proof of last subcase.

Lemma B.14.

If $Q \mid \Gamma \Vdash_n z \dot{\vdash} \tau \rightsquigarrow e \wedge \perp \notin e$ then also $Q \mid \Gamma \Vdash z \dot{\vdash} \tau \rightsquigarrow e$

Lemma B.15.

If $Q \mid \Gamma \vdash_n \sigma \sqsubseteq \tau \rightsquigarrow \bar{e} \wedge \perp \notin \bar{e}$ then also $Q \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$

Proof. (Of Lemma B.14) Induction on the structure of the derivation, where we have $Q \mid \Gamma \Vdash_n z \dot{\vdash} \tau \rightsquigarrow e$ (1) and $\perp \notin e$ (2).

Case $Q \mid \Gamma \Vdash_0 z \dot{\vdash} \tau \rightsquigarrow \perp$: by contradiction with (2).

Case $Q \mid \Gamma \Vdash_n z \dot{\vdash} \tau \rightsquigarrow z \bar{e}$

subcase $z : \sigma \in \Gamma$ (3) We have that only $[\text{VAR-DIRECTK}]$ is the only match, and we can derive $[\text{VAR-DIRECT}]$ via (3) and the inductive hypothesis.

subcase $z : \sigma \notin \Gamma$ (3) We have that only $[\text{VAR-QUALIFYK}]$ matches since (3) contradicts with the other subcase and thus $m/z : \sigma \in \Gamma$ (4), and we can derive $[\text{VAR-QUALIFY}]$ from (3,4) and the inductive hypothesis.

Proof. ((Of Lemma B.15)) Induction on the structure of the derivation where we have $Q \mid \Gamma \vdash_n \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ (1) and $\perp \notin \bar{e}$ (2). Trivial by case analysis on the derivation of $\perp \notin \bar{e}$ and passing n for the inductive hypotheses.

B.3 Principal Derivations

Here we prove Theorem 2.3: For any derivations $Q_1 \mid \Gamma \vdash_{\text{GEN}} e \dot{\vdash} \sigma \rightsquigarrow e_1$ and $Q_2 \mid \Gamma \vdash_{\text{GEN}} e \dot{\vdash} \sigma' \rightsquigarrow e_2$, we have $Q_1 = Q_2$, $\sigma = \sigma'$, and $e_1 = e_2$.

Leijen and Ye [2024] already show that type equivalence is principal (Theorem D.28):

Lemma B.16. (Principal type equivalence)

If $Q_1 \vdash \tau_1 \approx \tau_2$, then for any other derivation $Q_2 \vdash \tau_1 \approx \tau_2$, we have $Q_1 = Q_2$.

Next we show that variable resolution is principal (and coherent) as well:

Lemma B.17. (*Variable resolution is principal*)

If $Q_1 \mid \Gamma \vdash z : \tau \rightsquigarrow e_1$ and also $Q_2 \mid \Gamma \vdash z : \tau \rightsquigarrow e_2$, then $Q_1 = Q_2$ and $e_1 = e_2$.

Proof. For the elaboration, the equality $e_1 = e_2$ is by construction due to the $\exists!e$ premise. We still need to show though that we also have $Q_1 = Q_2$. Suppose we have $Q_1 \mid \Gamma \Vdash z : \tau \rightsquigarrow e$ and also $Q_2 \mid \Gamma \Vdash z : \tau \rightsquigarrow e$. We proceed by induction over the variable rules and instantiation:

Case [VAR-DIRECT]: by induction over the premise, for $Q_1 \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ and $Q_2 \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ we have $Q_1 = Q_2$.

Case [VAR-QUALIFY]: there is a choice here which $m/z : \sigma$ to pick from Γ . However, since they rewrite to the same $m/z \bar{e}$ both derivations use the same $m/z : \sigma$ and thus by induction over the instantiation premises, we have $Q_1 = Q_2$.

Case [INST-MONO]: We have $Q_1 \vdash \tau_1 \approx \tau_2$ and $Q_2 \vdash \tau_1 \approx \tau_2$, and by Lemma B.16, we have $Q_1 = Q_2$.

Case [INST-IMPLICIT]: We have $Q_{11} \mid \Gamma \vdash \rho \sqsubseteq \tau_2 \rightsquigarrow \bar{e}$ and $Q_{21} \mid \Gamma \vdash \rho \sqsubseteq \tau_2 \rightsquigarrow \bar{e}$ and by induction $Q_{11} = Q_{21}$ (I). We also have $Q_{12} \mid \Gamma \Vdash x : Q_{11}[\tau] \rightsquigarrow e_1$ and $Q_{22} \mid \Gamma \Vdash x : Q_{21}[\tau] \rightsquigarrow e_2$. However from (I) we have $Q_{11}[\tau] = Q_{21}[\tau]$, and thus by induction $e_1 = e_2$ and $Q_{12} = Q_{22}$, and thus $Q_{11}, Q_{12} = Q_{21}, Q_{22}$.

Case [INST-QUANTIFY]: From the premise we directly have by induction $Q_1 = Q_2$. □

With Lemma B.17 we can now follow the proof of Leijen and Ye [2024] (Appendix C) to show that the full rules are principal:

Proof. (*Of Theorem 2.3*) Since the rules in Figure 5 are syntax directed (with preference for [APP-VAR] over [APP-ARG]), the structure of the two derivations must match exactly. Furthermore, by Lemma B.16 and Lemma B.17, the premise $Q \vdash \tau_1 \approx \tau_2$ (in [ANN]) and the [VAR] rule are also principal. Therefore both derivations are exactly equal with no derivation choices. In particular, in a [VAR] leaf where $Q \mid \Gamma \vdash z : \tau \rightsquigarrow e'$ we always have the same τ for any derivation over $e[z]$, and it always elaborates to the same e' . □

B.4 Stability

First we show monomorphic substitution, then monomorphic stability, and finally polymorphic stability.

B.4.1 Monomorphic Substitution. Note: we first want to show other derivations exist and only later reason about unique derivations, so the lemmas are stated over $\Vdash$ for now. Moreover, since we need to reason about both the instance and variable rules together, we generally need to establish a stronger extended lemma first to have strong enough invariants.

The following Lemma is the inverse of stability (Lemma B.21) which we need in order to show that derivations are still unique in the stability theorem (Theorem 2.4).

Lemma B.18. (*Extended Monomorphic Substitution*)

If $Q_1 \mid \theta\Gamma \Vdash z : \theta\tau \rightsquigarrow e_1$, then for any $\tau' \sqsubseteq \tau$ (I) where $\tau = \theta'\tau'$ such that $\theta' \sqsubseteq (Q_1, \theta)$ (II), we also have $Q_2 \mid \Gamma \Vdash z : \tau' \rightsquigarrow e_2$ with $e_1 = e_2$ and $Q_2 \sqsubseteq (\theta, Q_1)$.

Lemma B.19. (*Monomorphic Substitution*)

If $Q_1 \mid \theta\Gamma \Vdash z : \theta\tau \rightsquigarrow e_1$, then we also have $Q_2 \mid \Gamma \Vdash z : \tau \rightsquigarrow e_2$ with $e_1 = e_2$ and $Q_2 \sqsubseteq (\theta, Q_1)$.

Proof. This follows directly from Lemma B.18 where $\tau' = \tau$. □

Note that in contrast to $\Vdash$, if $Q_1 \mid \theta\Gamma \vdash z \dot{\vdash} \theta\tau \rightsquigarrow e$, then we *cannot* conclude $Q_2 \mid \Gamma \vdash z \dot{\vdash} \tau \rightsquigarrow e$ also holds. In particular, we have by the premise $\exists!e. Q_1 \mid \theta\Gamma \Vdash z \dot{\vdash} \theta\tau \rightsquigarrow e$. By Lemma B.19, for all such derivations, we also have $Q_2 \mid \Gamma \Vdash z \dot{\vdash} \tau \rightsquigarrow e$ with the same e (satisfying the $\exists e$ part). However, we also need to show that there are no *other* derivations $Q_2 \mid \Gamma \Vdash z \dot{\vdash} \tau \rightsquigarrow e'$ with $e' \neq e$ (satisfying the unique part). Unfortunately, that is not generally the case. Consider the example from Section 2.10 where we have that $\emptyset \mid [\alpha:=c]\Gamma \vdash \text{trans} \dot{\vdash} [\alpha:=c](\alpha \rightarrow \alpha) \rightsquigarrow \text{generic/trans}$ holds, but $_ \mid \Gamma \not\vdash \text{trans} \dot{\vdash} \alpha \rightarrow \alpha$ does not as it is ambiguous.

Proof. (Of Lemma B.18) We proceed by induction over the variable and instance rules. For the instance rules, if $Q_1 \mid \theta\Gamma \vdash \theta\sigma \sqsubseteq \theta\tau \rightsquigarrow \overline{es}_1$, then also $Q_2 \mid \Gamma \vdash \sigma \sqsubseteq \tau' \rightsquigarrow \overline{es}_2$ with $e_1 = e_2$ and $Q_2 \sqsubseteq (\theta, Q_1)$.

Case [VAR-DIRECT]: By the premise, $z:\theta\sigma \in \theta\Gamma$ (1a) and $Q_1 \mid \theta\Gamma \Vdash \theta\sigma \sqsubseteq \theta\tau \rightsquigarrow e_1$ (1b). From (1a), we also have $z:\sigma \in \Gamma$. By induction on (1b), we have $Q_2 \mid \Gamma \Vdash \sigma \sqsubseteq \tau' \rightsquigarrow e_2$ with $e_1 = e_2$, and we can conclude $Q_2 \mid \Gamma \Vdash z \dot{\vdash} \tau' \rightsquigarrow e_2$.

Case [VAR-QUALIFY]: By the premise, $m/z:\theta\sigma \in \theta\Gamma$ (1a) and $Q_1 \mid \theta\Gamma \Vdash \theta\sigma \sqsubseteq \theta\tau \rightsquigarrow e_1$ (1b). From (1a), we also have $m/z:\sigma \in \Gamma$. By induction on (1b), we have $Q_2 \mid \Gamma \Vdash \sigma \sqsubseteq \tau' \rightsquigarrow e_2$ with $e_1 = e_2$, and we can conclude $Q_2 \mid \Gamma \Vdash m/z \dot{\vdash} \tau' \rightsquigarrow e_2$.

Case [INST-IMPLICIT]: We have $Q_{11} \mid \theta\Gamma \vdash \theta\rho \sqsubseteq \theta\tau \rightsquigarrow \overline{es}_1$ (1a) and $Q_{12} \mid \theta\Gamma \Vdash x \dot{\vdash} Q_{11}[\theta\tau_1] \rightsquigarrow e_1$ (1b). By induction on (1a), $Q_{21} \mid \Gamma \vdash \rho \sqsubseteq \tau' \rightsquigarrow \overline{es}_2$ (2a) with $\overline{es}_1 = \overline{es}_2$ (2b) and $Q_{21} \sqsubseteq (Q_{11}, \theta)$ (2c).

By Lemma B.23, $Q_{11} \circ \theta = (Q_{11}, \theta)$, and by composition, $= \theta \circ (Q_{11}, \theta)$. From (1b) we therefore have $Q_{12} \mid \theta\Gamma \vdash x \dot{\vdash} \theta((Q_{11}, \theta)[\tau_1]) \rightsquigarrow e_1$. By (2c), we also have $(\tau' \Rightarrow) Q_{21}[\tau_1] \sqsubseteq (Q_{11}, \theta)[\tau_1] (= \tau)$ (I) (and thus $(Q_{11}, \theta)[\tau_1] = (Q_{11}, \theta)[Q_{21}[\tau_1]]$ where $(Q_{11}, \theta) \sqsubseteq (Q_{11}, \theta)$ (II)). We can now use induction on (1b) to conclude $Q_{22} \mid \Gamma \vdash x \dot{\vdash} Q_{21}[\tau_1] \rightsquigarrow e_2$ (3a), with $e_1 = e_2$ (3b) and $Q_{22} \sqsubseteq (\theta, Q_{12})$ (3c). We can now derive from (2a,3a), $(Q_{21}, Q_{22}) \mid \Gamma \vdash ?x:\tau_1 \rightarrow \rho \sqsubseteq \tau' \rightsquigarrow e_2 \overline{es}_2$ with $e_1 \overline{es}_1 = e_2 \overline{es}_2$ (2b,3b), and $(Q_{21}, Q_{22}) \sqsubseteq ((Q_{11}, Q_{12}), \theta)$ (2c,3c).

Case [INST-MONO]: We have $Q_1 \vdash \theta\tau_1 \approx \theta\tau$ from the premise, and thus by Theorem B.25, $Q \vdash \tau_1 \approx \tau$ (1a) with $(Q, \theta) = (Q_1, \theta)$ (1b) and also $Q \sqsubseteq (Q_1, \theta)$ (1c). We can now derive:

$$\begin{aligned} & (Q_1, \theta)[\tau_1] \\ = & (Q_1, \theta)[Q[\tau_1]] \quad \{ (1c) \} \\ = & (Q_1, \theta)[Q[\tau]] \quad \{ (1a), \text{Theorem 2.1} \} \\ = & (Q_1, \theta)[\tau] \quad \{ (1c) \} \\ = & (Q_1, \theta)[\theta'\tau'] \quad \{ (I) \} \\ = & (Q_1, \theta)[\tau'] \quad \{ (II) \} \end{aligned}$$

And by Theorem 2.2, $Q_2 \vdash \tau_1 \approx \tau'$ with $Q_2 \sqsubseteq (Q_1, \theta)$. □

Case [INST-QUANTIFY]: We have $Q_1 \mid \theta\Gamma \vdash \theta\sigma \sqsubseteq \theta\tau \rightsquigarrow \overline{es}_1$. By induction, $Q_2 \mid \Gamma \vdash \sigma \sqsubseteq \tau' \rightsquigarrow \overline{es}_2$ with $\overline{es}_1 = \overline{es}_2$ and $Q_2 \sqsubseteq (Q_1, \theta)$. With the same fresh α , we can conclude $Q_2 \mid \Gamma \vdash \forall\alpha.\sigma \sqsubseteq \tau' \rightsquigarrow \overline{es}_2$.

B.4.2 Monomorphic Stability. To prove stability, we again first establish an extended lemma:

Lemma B.20. (Extended Monomorphic Stability)

If $Q_1 \mid \Gamma \Vdash z \dot{\vdash} \tau' \rightsquigarrow e_1$, and $\models (\theta, Q_1)$ (I), then for any $\tau' \sqsubseteq \tau$ (II) where $\tau = \theta'\tau'$ such that $\theta' \sqsubseteq (Q_1, \theta)$ (III), we have $Q_2 \mid \theta\Gamma \Vdash z \dot{\vdash} \theta\tau \rightsquigarrow e_2$ with $Q_1 \sqsubseteq (\theta, Q_2)$.

Lemma B.21. (Monomorphic Stability over Derivations)

If $Q_1 \mid \Gamma \Vdash z \dot{\vdash} \tau \rightsquigarrow e$ with $\models (\theta, Q_1)$, then we also have $Q_2 \mid \theta\Gamma \Vdash z \dot{\vdash} \theta\tau \rightsquigarrow e$ with $Q_1 \sqsubseteq (\theta, Q_2)$.

Proof. This follows directly from Lemma B.20 with $\tau' = \tau$. □

Proof. (Of Lemma B.20) We proceed by induction over the variable and instance rules. For the instance rules, if $Q_1 \mid \theta\Gamma \vdash \theta\sigma \sqsubseteq \theta\tau \rightsquigarrow \overline{es}_1$, then also $Q_2 \mid \Gamma \vdash \sigma \sqsubseteq \tau' \rightsquigarrow \overline{es}_2$ with $e_1 = e_2$ and $Q_2 \sqsubseteq (\theta, Q_1)$.

Case [VAR-DIRECT]: By the premise, $z : \sigma \in \Gamma$ (1a) and $Q_1 \mid \Gamma \Vdash \sigma \sqsubseteq \tau' \rightsquigarrow e_1$ (1b) with $\models (Q_1, \theta)$ (1c). From (1a), we also have $z : \theta\sigma \in \theta\Gamma$. By induction on (1b,1c), we have $Q_2 \mid \theta\Gamma \Vdash \theta\sigma \sqsubseteq \theta\tau \rightsquigarrow e_2$ with $e_1 = e_2$, and we can conclude $Q_2 \mid \theta\Gamma \Vdash z : \theta\tau \rightsquigarrow e_2$.

Case [VAR-QUALIFY]: By the premise, $m/z : \sigma \in \Gamma$ (1a) and $Q_1 \mid \Gamma \Vdash \sigma \sqsubseteq \tau' \rightsquigarrow e_1$ (1b) with $\models (Q_1, \theta)$ (1c). From (1a), we also have $m/z : \theta\sigma \in \theta\Gamma$. By induction on (1b,1c), we have $Q_2 \mid \theta\Gamma \Vdash \theta\sigma \sqsubseteq \theta\tau \rightsquigarrow e_2$ with $e_1 = e_2$, and we can conclude $Q_2 \mid \theta\Gamma \Vdash m/z : \theta\tau' \rightsquigarrow e_2$.

Case [INST-IMPLICIT]: We have $Q_{11} \mid \Gamma \vdash \rho \sqsubseteq \tau' \rightsquigarrow \overline{es}_1$ (1a), $Q_{12} \mid \Gamma \Vdash x : Q_{11}[\tau_1] \rightsquigarrow e_1$ (1b), and $\models (Q_{11}, Q_{12}, \theta)$ (1c). By induction on (1a,1c), $Q_{21} \mid \theta\Gamma \vdash \theta\rho \sqsubseteq \theta\tau \rightsquigarrow \overline{es}_2$ (2a) with $\overline{es}_1 = \overline{es}_2$ (2b) and $Q_{11} \sqsubseteq (Q_{21}, \theta)$ (2c).

By (2c), we also have $(\tau' =) Q_{11}[\tau_1] \sqsubseteq (Q_{21}, \theta)[\tau_1] (= \tau)$ (II), (and thus $(Q_{21}, \theta)[\tau_1] = (Q_{21}, \theta)[Q_{11}[\tau_1]]$ where $(Q_{21}, \theta) \sqsubseteq (Q_{21}, \theta)$ (III)).

By induction on (1b,1c) we now have $Q_{22} \mid \theta\Gamma \vdash x : \theta((Q_{21}, \theta)[\tau_1]) \rightsquigarrow e_2$ (3a), with $e_1 = e_2$ (3b) and $Q_{12} \sqsubseteq (\theta, Q_{22})$ (3c). By composition, $\theta \circ (Q_{21}, \theta) = (Q_{21}, \theta)$, and by Lemma B.23, $(Q_{21}, \theta) = Q_{21} \circ \theta$, and thus from (3a), $Q_{22} \mid \theta\Gamma \vdash x : Q_{21}[\theta\tau_1] \rightsquigarrow e_2$ (3d).

We can now derive from (2a,3d), $(Q_{21}, Q_{22}) \mid \theta\Gamma \vdash \theta(?x:\tau_1 \rightarrow \rho) \sqsubseteq \theta\tau \rightsquigarrow e_2 \overline{es}_2$ with $e_1 \overline{es}_1 = e_2 \overline{es}_2$ (2b,3b), and $(Q_{11}, Q_{12}) \sqsubseteq ((Q_{21}, Q_{22}), \theta)$ (2c,3c).

Case [INST-MONO]: We have $Q_1 \vdash \tau_1 \approx \tau'$ (1a) from the premise with $\models (Q_1, \theta)$. We can now derive:

$$\begin{aligned}
& (Q_1, \theta)[\theta\tau_1] \\
= & (Q_1, \theta)[\tau_1] \quad \{ \theta \sqsubseteq (Q_1, \theta) \} \\
= & (Q_1, \theta)[Q_1[\tau_1]] \quad \{ Q_1 \sqsubseteq (Q_1, \theta) \} \\
= & (Q_1, \theta)[Q_1[\tau']] \quad \{ (1a), \text{Theorem 2.1} \} \\
= & (Q_1, \theta)[\tau'] \quad \{ \} \\
= & (Q_1, \theta)[\theta'\tau'] \quad \{ (III) \} \\
= & (Q_1, \theta)[\tau] \quad \{ \text{def.} \} \\
= & (Q_1, \theta)[\theta\tau] \quad \{ \theta \sqsubseteq (Q_1, \theta) \}
\end{aligned}$$

And by Theorem 2.2, $Q_2 \vdash \theta\tau_1 \approx \theta\tau$ with $Q_2 \sqsubseteq (Q_1, \theta)$.

Case [INST-QUANTIFY]: We have from the premise $Q_1 \mid \Gamma \vdash \sigma \sqsubseteq \tau' \rightsquigarrow \overline{es}_1$ with $\models (\theta, Q_1)$. By induction, $Q_2 \mid \theta\Gamma \vdash \theta\sigma \sqsubseteq \theta\tau \rightsquigarrow \overline{es}_2$ with $\overline{es}_1 = \overline{es}_2$ and $Q_1 \sqsubseteq (Q_2, \theta)$. With the same fresh α , we can derive $Q_2 \mid \theta\Gamma \vdash \theta(\forall\alpha.\sigma) \sqsubseteq \theta\tau \rightsquigarrow \overline{es}_2$.

□

Now we are able to establish stability over variable resolution: if $Q_1 \mid \Gamma \vdash z : \tau \rightsquigarrow e$ with $\models (\theta, Q_1)$ (I), then we also have $Q_2 \mid \theta\Gamma \vdash z : \theta\tau \rightsquigarrow e$ with $Q_1 \sqsubseteq (\theta, Q_2)$.

Proof. (Of Theorem 2.4) From the premise of [VAR], we have $\exists!e$ (1a) with $Q_1 \mid \Gamma \Vdash z : \tau \rightsquigarrow e$ (1b). By Theorem B.21 (1a,1b), we also have $Q_2 \mid \theta\Gamma \Vdash z : \theta\tau \rightsquigarrow e$ (2a) with $Q_1 \sqsubseteq (Q_2, \theta)$ (2b) satisfying $\exists e$. Now, we also need to show there are no *other* derivations $Q \mid \theta'\Gamma \Vdash z : \theta'\tau \rightsquigarrow e'$ for some θ' with $e \neq e'$. However, if such derivation exists, we also have by Lemma B.19, $Q' \mid \Gamma \Vdash z : \tau \rightsquigarrow e'$ – but that contradicts (1a,1b). □

B.4.3 Polymorphic Stability. We prove Theorem 2.5: If using the [ANN-SCHEME] rule, we have $Q \mid \Gamma \vdash t : \forall\bar{\alpha}.\rho \rightsquigarrow e$, and for any $\bar{\tau}$ with $\text{ftv}(\bar{\tau}) \subseteq \text{ftv}(\Gamma)$ (I), then also $Q' \mid \Gamma \vdash t : [\bar{\alpha} := \bar{\tau}]\rho \rightsquigarrow e$.

Proof. (Of Theorem 2.5) By the premise, $Q_1 \mid \Gamma, \bar{\alpha} \vdash t : \rho \rightsquigarrow e$ (1a), with $(Q, \forall\bar{\alpha}.\rho) = \text{gen}(Q_1, \Gamma, \rho)$ (1b) and fresh $\bar{\alpha}$ (1c). By (1b), we must have $\bar{\alpha} \notin \text{dom}(Q_1)$ (1d) and $\bar{\alpha} \notin \text{ftv}(Q)$.

We can now show that we can also derive $Q' \mid \Gamma \vdash t \vdash [\bar{\alpha} := \bar{\tau}] \rho \rightsquigarrow e$ by exactly following the derivation of (1a). The interesting cases are at the $[\text{VAR}]$ leaf nodes. In the original derivation, we have a leaf $Q_3 \mid \Gamma, \bar{\alpha}, \Gamma' \vdash z \vdash \tau \rightsquigarrow e_3$ (2a) and in the new one we need $Q_4 \mid \Gamma[\bar{\alpha} := \bar{\tau}] \Gamma' \vdash z \vdash [\bar{\alpha} := \bar{\tau}] \tau \rightsquigarrow e_3$. We must have $\bar{\alpha} \notin \text{dom}(Q_3)$ by (1d), and therefore $\models ([\bar{\alpha} := \bar{\tau}], Q_3)$ (2b). We can now use Theorem 2.4 with (2a, 2b), to derive $Q_4 \mid [\bar{\alpha} := \bar{\tau}] \Gamma, \bar{\alpha}, \Gamma' \vdash z \vdash [\bar{\alpha} := \bar{\tau}] \tau \rightsquigarrow e_3$ (3a). Since the $\bar{\alpha}$ bindings are anonymous, and fresh $\bar{\alpha}$, we have $Q_4 \mid \Gamma, [\bar{\alpha} := \bar{\tau}] \Gamma' \vdash z \vdash [\bar{\alpha} := \bar{\tau}] \tau \rightsquigarrow e_3$ (3b).

(As an aside, this is the situation where using skolem constants are unstable: with skolems, we may originally have a leaf node in type equivalence as $\not\vdash \tau' \approx \tau[\bar{\alpha} := \bar{c}]$. But if we later instantiate the $\bar{\alpha}$, we get $\vdash \tau' \approx [\bar{\alpha} := \bar{\tau}]$ and now this may hold, which can lead to a new valid derivation, and later on the $\exists!e$ condition in $[\text{VAR}]$ cannot be satisfied.) $\square$

B.4.4 Supporting Lemmas.

Lemma B.22. (*Prefix composition can be composed*)

If $\text{ftv}(Q_1) \not\cap \text{dom}(Q_2)$, then $(Q_1, Q_2) = Q_1 \circ Q_2$.

Lemma B.23.

If $Q \mid \theta \Gamma \vdash \theta \sigma \sqsubseteq \theta \tau \rightsquigarrow \bar{e}$, then $\text{ftv}(Q) \not\cap \text{dom}(\theta)$, and by Lemma B.22, $(Q, \theta) = Q \circ \theta$.

Lemma B.24.

If $Q \mid \theta \Gamma \vdash \theta \sigma \sqsubseteq \theta \tau \rightsquigarrow \bar{e}$, then also $Q' \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}'$

We have [Leijen and Ye 2024, D.27]:

Theorem B.25. (*Type equivalent substitution*)

If $Q_1 \vdash Q[\tau_1] \approx Q[\tau_2]$, then also $Q_2 \vdash \tau_1 \approx \tau_2$ with $(Q, Q_1) = (Q, Q_2)$.

and also [Leijen and Ye 2024, Lemma 3.9]:

Lemma B.26. (*Extraction corresponds to composition of prefix solutions*)

If $\models Q$ and $Q = Q' \cdot \alpha = \tau$, then $\langle Q \rangle = \langle Q' \rangle \circ [\alpha := \tau]$.

where

$$\frac{Q = Q' \cup \{\alpha = \tau\} \quad \alpha \notin \text{ftv}(Q', \tau)}{Q = Q' \cdot \alpha = \tau} \text{EXTRACT}$$

We can also simplify duplicate bindings [Leijen and Ye 2024, Theorem 2.7]:

Theorem B.27. (*Simplify*)

If $Q' \vdash \tau_1 \approx \tau_2$, then $Q \cup \{\alpha = \tau_1, \alpha = \tau_2\} = Q \cup Q' \cup \{\alpha = \tau_1\}$

C UNIFICATION AND PREFIX COMPOSITION

Leijen and Ye [2025] show how to implement prefix composition (*compose*) and type equivalence (*unify*). Unfortunately, in the original publication their formulation is not technically complete unless we assume prefixes are always *well-formed*.

The reason for this is rather subtle, and requires careful reasoning about equivalent substitutions. We say two substitutions θ_1, θ_2 are equivalent whenever each is an instance of the other: $\theta_1 \sqsubseteq \theta_2 \wedge \theta_2 \sqsubseteq \theta_1$ (where $\theta \sqsubseteq \theta'$ iff $\theta' = \theta'' \circ \theta$ for some θ''). This means that a substitution from a type variable to another has no direction: $[\alpha := \beta] \equiv [\beta := \alpha]$ since each is an instance of the other.

Since equality of prefixes is defined as equivalence of the minimal solutions, this is also the case for prefixes, where $\{\alpha = \beta\} = \{\beta = \alpha\}$. For the *solve* algorithm, we need to define an ordering on such type variable equalities in order to make it easier to correctly detect cycles. We assume there is some lexical ordering of type variables such that for every type variable equality $\alpha = \beta$, we have

$$\begin{aligned}
& \text{unify} : (\tau, \tau) \rightarrow \theta \\
& \text{unify}(\alpha, \alpha) = \text{id} \\
& \text{unify}(\alpha, \beta) \quad | \alpha \neq \beta = \text{if } \alpha < \beta \text{ then } [\alpha := \beta] \text{ else } [\beta := \alpha] \\
& \text{unify}(\alpha, \tau) \text{ or } (\tau, \alpha) \quad | \alpha \notin \text{ftv}(\tau) = [\alpha := \tau] \\
& \text{unify}(\tau_1 \rightarrow \tau_2, \tau'_1 \rightarrow \tau'_2) = \text{let } \theta_1 = \text{unify}(\tau_1, \tau'_1); \theta_2 = \text{unify}(\tau_2, \tau'_2) \text{ in solve}(\theta_1 \cup \theta_2) \\
& \text{unify}(_, _) = \text{fail}() \\
\\
& \text{solve} : Q \rightarrow \theta \\
& \text{solve}(\emptyset) = \text{id} \\
& \text{solve}(Q \uplus \{\alpha = \tau\}) = \text{solve}(Q) \circ [\alpha := \tau] \quad \text{if } \alpha \notin \text{ftv}(Q, \tau) \\
& \text{solve}(Q \uplus \{\alpha = \tau_1, \alpha = \tau_2\}) = \text{solve}(Q \cup Q' \cup \{\alpha = \tau_1\}) \quad \text{if } Q' = \text{unify}(\tau_1, \tau_2) \wedge \alpha \notin \text{ftv}(\tau_1, \tau_2, \text{rng}(Q)) \\
& \text{solve}(_) = \text{fail}() \\
\\
& \text{compose} : (Q, Q) \rightarrow \theta \\
& \text{compose}(Q_1, Q_2) = \text{solve}(Q_1 \cup Q_2)
\end{aligned}$$

Fig. 7. Unification and solving of prefixes (where we use $\uplus$ for disjoint union).

that $\alpha < \beta$ (and $\beta = \alpha$ is never present). Moreover, we also assume we never have $\alpha = \alpha$ constraints. We call such prefixes well formed.

We can easily ensure well-formed prefixes by adapting the *unify* algorithm [Leijen and Ye 2025] to include one extra case for type variable equalities, which ensures the constraint $\alpha = \beta$ is always in the correct order with $\alpha < \beta$. The full rules for *unify*, *solve*, and *compose* are given in Figure 7.

Essentially the *solve* algorithm picks non-dependent bindings and composes them recursively, while simplifying duplicate bindings away by unifying their types using the *unify* function. Note that the extra case for variable equalities also ensures now that $\text{unify}(\tau_1, \tau_2) = \text{unify}(\tau_2, \tau_1)$.

As an aside, the need for well-formed prefixes can be illustrated by using the ill-formed prefix $\{\beta = \alpha, \alpha = \beta, \alpha = \text{int}\}$. In such case *solve* would fail while there actually exists a valid substitution (i.e. *solve* is incomplete for ill-formed prefixes). In contrast, the well-formed equivalent $\{\alpha = \beta, \alpha = \beta, \alpha = \text{int}\}$ solves indeed to $[\alpha := \text{int}, \beta := \text{int}]$.

C.1 Soundness

Lemma C.28. (*Unification is Sound*)

If $\langle Q \rangle = \text{unify}(\tau_1, \tau_2)$, then $Q \vdash \tau_1 \approx \tau_2$.

Lemma C.29. (*Solve is sound*)

If $\theta = \text{solve}(Q)$ (with a well-formed Q), then $\models Q$ and $\theta = \langle Q \rangle$.

We establish soundness of *unify* and *solve* together, since they are mutually recursive.

Proof. (Of Lemma C.28 and Lemma C.29) By induction on the rules of *unify* and *solve*.

Case $\text{unify}(\alpha, \alpha)$: we have $\langle Q \rangle = \text{id}$ and thus $Q = \emptyset$. By [EQ-ID], we have $\emptyset \vdash \alpha \approx \alpha$.

Case $\text{unify}(\alpha, \beta)$ with $\alpha \neq \beta$: With $\alpha < \beta$, we have $Q = \{\alpha = \beta\}$, and by [EQ-VAR] we have $Q \vdash \alpha \approx \beta$. For $\beta < \alpha$, we have $Q = \{\beta = \alpha\}$, and by [EQ-VAR] and [EQ-REFL], we also have $Q \vdash \beta \approx \alpha$.

Case $\text{unify}(\alpha, \tau), \alpha \notin \text{ftv}(\tau)$: we have $Q = \{\alpha = \tau\}$, and by [EQ-VAR] we have $Q \vdash \alpha \approx \tau$.

Case $\text{unify}(\tau, \alpha), \alpha \notin \text{ftv}(\tau)$: we have $Q = \{\alpha = \tau\}$, and by [EQ-VAR] and [EQ-REFL], we have $Q \vdash \tau \approx \alpha$.

Case $\text{unify}(\tau_1 \rightarrow \tau_2, \tau'_1 \rightarrow \tau'_2)$: From the premises, we have $Q_1 = \text{unify}(\tau_1, \tau'_1)$ (1a), $Q_2 = \text{unify}(\tau_2, \tau'_2)$ (1b), and $Q = \text{solve}(Q_1 \cup Q_2)$ (1c). By induction over (1a,1b), we also have $Q_1 \vdash \tau_1 \approx \tau'_1$ and $Q_2 \vdash \tau_2 \approx \tau'_2$.

Moreover, by induction on $\text{solve}(Q_1 \cup Q_2)$, we have $\models (Q_1 \cup Q_2)$ with $\langle (Q_1, Q_2) \rangle = \text{solve}(Q_1, Q_2)$. We can now use [EQ-FUN] to derive $(Q_1, Q_2) \vdash \tau_1 \rightarrow \tau_2 \approx \tau'_1 \rightarrow \tau'_2$.

Case $\text{unify}(\tau_1, \tau_2) = \text{fail}()$: in this case the precondition is not satisfied.

For the rules of solve , we have:

Case $\text{solve}(\emptyset)$: we have trivially $\models \emptyset$, and $\theta = \text{id} = \langle \emptyset \rangle$.

Case $\text{solve}(Q \uplus \{\alpha = \tau\})$ with $\alpha \notin \text{ftv}(Q)$ (1a): we have $\theta = \text{solve}(Q) \circ [\alpha := \tau]$ (1b). By induction on $\text{solve}(Q)$, we have $\text{solve}(Q) = \langle Q \rangle$ (2a) with $\models Q$ (2b). By (1a, 2b), we also have $\models (Q \cup \{\alpha = \tau\})$. By Theorem B.26, we have $\langle (Q \cup \{\alpha = \tau\}) \rangle = \langle Q \rangle \circ [\alpha := \tau]$ (3). Therefore

$$\begin{aligned} & \theta \\ &= \text{solve}(Q) \circ [\alpha := \tau] \quad \{ (1b) \} \\ &= \langle Q \rangle \circ [\alpha := \tau] \quad \{ (2a) \} \\ &= \langle Q \cup \{\alpha = \tau\} \rangle \quad \{ (3) \} \end{aligned}$$

Case $\text{solve}(Q \uplus \{\alpha = \tau_1, \alpha = \tau_2\})$: we have $\langle Q' \rangle = \text{unify}(\tau_1, \tau_2)$ (1a) and $\theta = \text{solve}(Q \cup Q' \cup \{\alpha = \tau_1\})$ (1b). By induction on (1a), $Q' \vdash \tau_1 \approx \tau_2$ (2a), and by induction on (1b), $\models (Q \cup Q' \cup \{\alpha = \tau_1\})$ (2b) where $\theta = \langle (Q \cup Q' \cup \{\alpha = \tau_1\}) \rangle$ (2c). From Theorem B.27 and (2a), we also have $(Q \cup \{\alpha = \tau_1\} \cup \{\alpha = \tau_2\}) = (Q \cup Q' \cup \{\alpha = \tau_1\})$, and thus $\theta = \langle (Q \cup \{\alpha = \tau_1\} \cup \{\alpha = \tau_2\}) \rangle$ (2c).

Case $\text{solve}(Q) = \text{fail}()$: in this case the precondition is not satisfied.

□

C.2 Completeness

Lemma C.30. (*Unify is complete*)

If $Q \vdash \tau_1 \approx \tau_2$, then $\langle Q \rangle = \text{unify}(\tau_1, \tau_2)$.

Lemma C.31. (*Solve is complete*)

If $\models Q$ (with a well-formed Q), then $\theta = \text{solve}(Q)$ with $\theta = \langle Q \rangle$.

Again, we establish completeness of unify and solve together since they are mutually recursive. For clarity, we split the proofs and assume completeness of the other in each proof.

Proof. (Of Lemma C.30) We proceed by induction over the derivation (assuming solve is complete):

Case [EQ-ID]: We have $\emptyset \vdash \tau_1 \approx \tau_2$ with $\tau_1 = \tau_2$. If we only have function arrows and variables, we can repeatedly apply [EQ-FUN] (with $Q_1 = Q_2 = \emptyset$), ending in $\emptyset \vdash \alpha \approx \alpha$. In that case we have $\text{unify}(\alpha, \alpha) = \text{id}$ (with $\text{id} = \langle \emptyset \rangle$). For each [EQ-FUN] we can apply the $\text{unify}(\tau_1 \rightarrow \tau_2, \tau_3 \rightarrow \tau_4)$ rules where $\text{solve}(\emptyset, \emptyset) = \text{id}$ again.

Case [EQ-VAR]: We have $\{\alpha = \tau\} \vdash \alpha \approx \tau$ (1a) with $\alpha \notin \text{ftv}(\tau)$ (1b). Suppose $\tau = \beta$ (with $\alpha \neq \beta$ (1b)). If $\alpha < \beta$, $\text{unify}(\alpha, \beta) = [\alpha := \beta] = \langle \{\alpha = \beta\} \rangle$. If $\beta < \alpha$, we have $\text{unify}(\alpha, \beta) = [\beta := \alpha]$ but that equals also $\beta := \alpha$ and $= \langle \{\alpha = \beta\} \rangle$. Otherwise, $\tau \neq \beta$, and by (1b), we have $\text{unify}(\alpha, \tau) = [\alpha := \tau] = \langle \{\alpha = \tau\} \rangle$.

Case [EQ-REFL]: We have $Q \vdash \tau_2 \approx \tau_1$ by the premise, and by induction $\text{unify}(\tau_2, \tau_1) = \langle Q \rangle$. We have $\text{unify}(\tau_2, \tau_1) = \text{unify}(\tau_1, \tau_2)$, and therefore we also have $\langle Q \rangle = \text{unify}(\tau_1, \tau_2)$.

Case [EQ-FUN]: We have $Q_1 \vdash \tau_1 \approx \tau_3$ (1a) and $Q_2 \vdash \tau_2 \approx \tau_4$ (1b) by the premises. The $\text{unify}(\tau_1 \rightarrow \tau_2, \tau_3 \rightarrow \tau_4)$ case applies, and we have by induction over (1a, 1b), $\langle Q_1 \rangle = \text{unify}(\tau_1, \tau_3)$ and $\langle Q_2 \rangle = \text{unify}(\tau_2, \tau_4)$, and by Lemma C.31, $\langle (Q_1, Q_2) \rangle = \text{solve}(Q_1, Q_2)$.

Case If no derivation rules apply, we also have that none of the previous unify cases apply, and we have $\text{unify}(\tau_1, \tau_2) = \text{fail}()$.

□

For showing completeness of *solve*, we also need to establish a decreasing measure for the induction.

For well-formed prefixes, we can define a stable *degree* of a prefix (which decreases in each recursive step). First we define the dependencies of α with respect to a Q as:

- $\beta \in \text{deps}(\alpha)$ if $\alpha = \tau \in Q$ and $\beta \in \text{ftv}(\tau)$.
- $\gamma \in \text{deps}(\alpha)$ if $\beta \in \text{deps}(\alpha)$ and $\gamma \in \text{deps}(\beta)$ (transitive closure).

We say α is independent of β , $\alpha \prec \beta$, iff $\alpha \notin \text{deps}(\beta)$ (where we have both $\alpha \prec \beta$ and $\beta \prec \alpha$ for independent type variables). The degree of Q is now the number of occurrences of distinct type variables in the domain ordered by $\prec$ (in some order). For example, $\text{degree}(\{\beta = \gamma, \alpha = \text{int} \rightarrow \beta, \gamma = \text{int}, \alpha = \beta \rightarrow \text{int}\}) = (2, 1, 1)$ (for (α, β, γ)). We have that if $\alpha \notin \text{codom}(Q)$, then $\forall \beta \in \text{dom}(Q). \alpha \prec \beta$ (I).

Proof. (Of Lemma C.31) We proceed by induction on the degree and shape of Q :

Case $Q = \emptyset$: in that case $\text{solve}(\emptyset) = \text{id}$ applies where $\text{id} = \langle \emptyset \rangle$.

Otherwise $Q \neq \emptyset$. Suppose we have that for all $\alpha \in \text{dom}(Q)$ that $\alpha \in \text{codom}(Q)$.

Since all domain variables appear in the co-domain, by the pigeon hole principle such prefix must contain a cycle, for example, $\{\alpha = \text{int}, \beta = \alpha \rightarrow \gamma, \gamma = \beta \rightarrow \beta\}$. Since we order type variable equalities in a well-formed prefix, it is not possible for such cycle to consist of only type variable equalities (like $\{\alpha = \beta, \beta = \alpha\}$). Therefore, in such cycle there must be some $\alpha = \tau$ with τ being a larger type (e.g. $\tau_1 \rightarrow \tau_2$). However, that makes it no longer possible to create an idempotent substitution and $\neq Q$, contradicting the assumption.

Therefore, we know there must be at least some $\alpha \in \text{dom}(Q)$ where $\alpha \notin \text{codom}(Q)$ (1). We proceed now by case analysis:

Case $Q = Q' \uplus \{\alpha = \tau\}$ with $\alpha \notin \text{dom}(Q')$. In that case, with (1) we have $\alpha \notin \text{ftv}(Q', \tau)$ (2) and the second case of *solve* applies. By induction on the decreasing degree, $\langle Q' \rangle = \text{solve}(Q')$ (3). By (2) and [EXTRACT], we have $Q = Q' \cdot \{\alpha = \tau\}$ and by Lemma B.26, $\langle Q \rangle = \langle Q' \rangle \circ [\alpha := \tau]$. With (3) we now have $\langle Q \rangle = \text{solve}(Q') \circ [\alpha := \tau]$.

Case Otherwise, we must have $Q = Q' \uplus \{\alpha = \tau_1, \alpha = \tau_2\}$. With (1), we have $\alpha \notin \text{ftv}(\tau_1, \tau_2, \text{rng}(Q))$ (1a), and the third case of *solve* applies. Since $\models Q$, by simplification (Theorem B.27), we have $Q' \cup \{\alpha = \tau_1, \alpha = \tau_2\} = Q' \cup Q'' \cup \{\alpha = \tau_1\}$ (2a), with $Q'' \vdash \tau_1 \approx \tau_2$ (2b).

From the completeness of *unify*, and (2b) we have $\langle Q'' \rangle = \text{unify}(\tau_1, \tau_2)$ (3). By (2b) $\text{ftv}(Q'') \subseteq \text{ftv}(\tau_1, \tau_2)$, and thus by (1a), we have one less $\alpha \in \text{dom}(Q' \cup Q'' \cup \{\alpha = \tau_1\})$. Moreover, since $\alpha \notin \text{codom}(Q' \cup Q'')$ and (I), the degree decreases, and by induction $\langle Q \rangle = \text{solve}(Q' \cup Q'' \cup \{\alpha = \tau_1\})$.

Since this covers all forms of Q with $\models Q$, the fail case never applies. $\square$

D SOUNDNESS OF THE TYPE RULES

Theorem D.32. (Soundness of the Type Rules)

If $Q \mid \Gamma \vdash_{\text{GEN}} e \overset{\tau}{\rightarrow} \sigma \rightsquigarrow e'$, then also $Q \mid \llbracket \Gamma \rrbracket \vdash_{\text{LY}} e' : \llbracket \sigma \rrbracket$ in the type rules of Leijen and Ye [2025]. Similarly, if $Q \mid \Gamma \vdash e \overset{\tau}{\rightarrow} \tau \rightsquigarrow e'$, then also $Q \mid \llbracket \Gamma \rrbracket \vdash_{\text{LY}} e' : \llbracket \tau \rrbracket$, and if $Q \mid \Gamma \vdash e \overset{\tau}{\rightarrow} \tau \rightsquigarrow e'$, then also $Q \mid \llbracket \Gamma \rrbracket \vdash_{\text{LY}} e' : \llbracket \tau \rrbracket$.

where we define:

$$\begin{aligned} \llbracket \tau \rrbracket &= \tau \\ \llbracket x : \tau \rightarrow \rho \rrbracket &= \tau \rightarrow \llbracket \rho \rrbracket \\ \llbracket \forall \alpha. \sigma \rrbracket &= \forall \alpha. \llbracket \sigma \rrbracket \end{aligned}$$

$$\llbracket z_1 : \sigma_1, \dots, z_n : \sigma_n \rrbracket = z_1 : \llbracket \sigma_1 \rrbracket, \dots, z_n : \llbracket \sigma_n \rrbracket$$

to translate our types to regular Hindley-Milner types.

Lemma D.33. (*Generalization is Sound*)

If $(Q_0, \sigma) = \text{gen}(Q, \Gamma, \rho)$, and $Q \mid \llbracket \Gamma \rrbracket \vdash e : \llbracket \rho \rrbracket$, then we can also derive $Q_0 \mid \llbracket \Gamma \rrbracket \vdash e : \llbracket \sigma \rrbracket$ (for any e).

Lemma D.34. (*Variable instantiation is Sound*)

If $Q \mid \Gamma \vdash z : \tau \rightsquigarrow e'$, then also $Q \mid \Gamma \vdash e' : \tau$, and furthermore, if $Q \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$, then then for any $Q' \mid \llbracket \Gamma \rrbracket \vdash e' : \llbracket \sigma \rrbracket$, we have $(Q', Q) \mid \llbracket \Gamma \rrbracket \vdash e' \bar{e} : \llbracket \tau \rrbracket$.

Proof. (of Theorem D.32) By induction over the type derivation.

Case [GEN]: By induction on the left premise, we have $Q_0 \mid \llbracket \Gamma \rrbracket \vdash t' : \llbracket \rho \rrbracket$. Moreover, we have $(Q, \sigma) = \text{gen}(Q_0, \Gamma, \rho)$, and by Lemma D.33, $Q \mid \llbracket \Gamma \rrbracket \vdash t' : \llbracket \sigma \rrbracket$.

Case [VAR]: By the premise, we have a unique $Q \mid \Gamma \vdash z : \tau \rightsquigarrow e'$. and thus by Lemma D.34 we have $Q \mid \llbracket \Gamma \rrbracket \vdash e' : \tau$ as well.

All other rules follow directly by induction as well. $\square$

Proof. (Of Lemma D.34) By induction over the type derivation (including instantiation).

Case [var – direct]: We have $z : \sigma \in \Gamma$ (1) and $Q \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ (2). By induction on (2), we have for any $Q' \mid \llbracket \Gamma \rrbracket \vdash e : \llbracket \sigma \rrbracket$ that $(Q', Q) \mid \llbracket \Gamma \rrbracket \vdash e \bar{e} : \llbracket \tau \rrbracket$ (3). By [VAR] and (1), we have $\emptyset \mid \llbracket \Gamma \rrbracket \vdash z : \llbracket \sigma \rrbracket$, and by (3), $Q \mid \llbracket \Gamma \rrbracket \vdash z \bar{e} : \llbracket \tau \rrbracket$.

Case [var – qualify]: We have $m/z : \sigma \in \Gamma$ (1) and $Q \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ (2). By induction on (2), we have for any $Q' \mid \llbracket \Gamma \rrbracket \vdash e : \llbracket \sigma \rrbracket$ that $(Q', Q) \mid \llbracket \Gamma \rrbracket \vdash e \bar{e} : \llbracket \tau \rrbracket$ (3). By [VAR] and (1), we have $\emptyset \mid \llbracket \Gamma \rrbracket \vdash m/z : \llbracket \sigma \rrbracket$, and by (3), $Q \mid \llbracket \Gamma \rrbracket \vdash m/z \bar{e} : \llbracket \tau \rrbracket$.

Case [inst – mono]: We have $Q \vdash \tau_1 \approx \tau_2$. Since $\llbracket \tau \rrbracket = \tau$, we also have $Q \vdash \llbracket \tau_1 \rrbracket \approx \llbracket \tau_2 \rrbracket$ (1). For any $Q' \mid \llbracket \Gamma \rrbracket \vdash e : \llbracket \tau_1 \rrbracket$, we can use (1) and [CHK] to derive $(Q', Q) \mid \llbracket \Gamma \rrbracket \vdash e : \llbracket \tau_2 \rrbracket$.

Case [INST-IMPLICIT]: We $Q_1 \mid \Gamma \vdash \rho \sqsubseteq \tau_2 \rightsquigarrow \bar{e}'$ (1a) and $Q_2 \mid \Gamma \vdash x : Q_1[\tau_1] \rightsquigarrow e'$ (1b). By induction on (1a), we also have for any $Q_0 \mid \llbracket \Gamma \rrbracket \vdash e_0 : \llbracket \rho \rrbracket$ that $(Q_0, Q_1) \mid \llbracket \Gamma \rrbracket \vdash e_0 \bar{e}' : \llbracket \tau_2 \rrbracket$ (2a), and $Q_2 \mid \llbracket \Gamma \rrbracket \vdash e' : \llbracket Q_1[\tau_1] \rrbracket$, and thus $Q_2 \mid \llbracket \Gamma \rrbracket \vdash e' \bar{e}' : Q_1[\tau_1]$ (2b). By Lemma D.35, we also have $Q \mid \llbracket \Gamma \rrbracket \vdash e' : \tau_1$ (3a) where $(Q_1, Q_2) = (Q_1, Q)$ (3b).

Now for any $Q' \mid \llbracket \Gamma \rrbracket \vdash e : (x : \tau_1) \rightarrow \rho$, we can derive $(Q', Q) \mid \llbracket \Gamma \rrbracket \vdash e e' : \llbracket \rho \rrbracket$ from (3a, [APP]), and thus $(Q', Q, Q_1) \mid \llbracket \Gamma \rrbracket \vdash e e' \bar{e}' : \llbracket \tau_2 \rrbracket$ from (2a), and by (3b) $(Q', Q_1, Q_2) \mid \llbracket \Gamma \rrbracket \vdash e e' \bar{e}' : \llbracket \tau_2 \rrbracket$.

Case [INST-QUANTIFY]: We have $Q \mid \Gamma \vdash \sigma \sqsubseteq \tau \rightsquigarrow \bar{e}$ with fresh α (1), and by induction for any $Q_0 \mid \Gamma' \vdash e_0 : \sigma$, we have $(Q_0, Q) \mid \llbracket \Gamma \rrbracket \vdash e_0 \bar{e} : \llbracket \tau \rrbracket$ (2). For any $Q' \mid \Gamma' \vdash e : \forall \alpha. \sigma$, we can use [INST] with (1) to derive $Q' \mid \Gamma' \vdash e : \sigma$ and thus by (2), $(Q', Q) \mid \llbracket \Gamma \rrbracket \vdash e \bar{e} : \llbracket \tau \rrbracket$ as well. $\square$

Lemma D.35. (*Derivation Substitution*)

If $Q_1 \mid \Gamma \vdash e : Q[\tau]$ and $\models (Q, Q_1)$, then also $Q_2 \vdash e : \tau$ where $(Q, Q_1) = (Q, Q_2)$

Proof. By [CHK], we have $Q_3 \mid \Gamma \vdash e : \tau'$ and $Q_4 \vdash \tau' \approx Q[\tau]$ with $Q_1 = (Q_3, Q_4)$, and $\models (Q, Q_3, Q_4)$.

Therefore, $Q_5 \vdash \tau' \approx \tau$ with $(Q, Q_5) = (Q, Q_4)$, and thus $(Q_3, Q_5) \vdash e : \tau$ with $(Q, Q_1) = (Q, Q_3, Q_4) = (Q, Q_3, Q_5)$. $\square$

Proof follows from Lemma D.27 [Leijen and Ye 2024]

